

DSM TP 2017

8th International Summer School
on Domain-Specific Modeling
Theory and Practice

Montreal, Canada
10-14 July 2017

Variability / Product Families

Kacper Bąk

Currently: The MathWorks, Inc.
Previously: GSD Lab, University of Waterloo

Acknowledgments

slides based on tutorials by

Andrzej Wąsowski, IT University of Copenhagen

Michał Antkiewicz, University of Waterloo

Krzysztof Czarnecki, University of Waterloo





Software-intensive Products Come in Many Variants





Domain Engineering

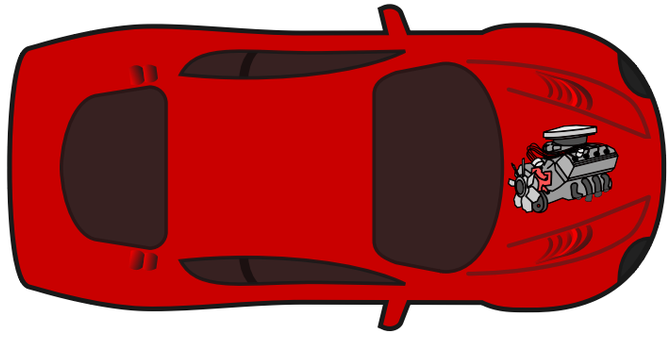
aka Product Line Engineering

aka Product Family Engineering

Application Engineering

Development *with* Reuse

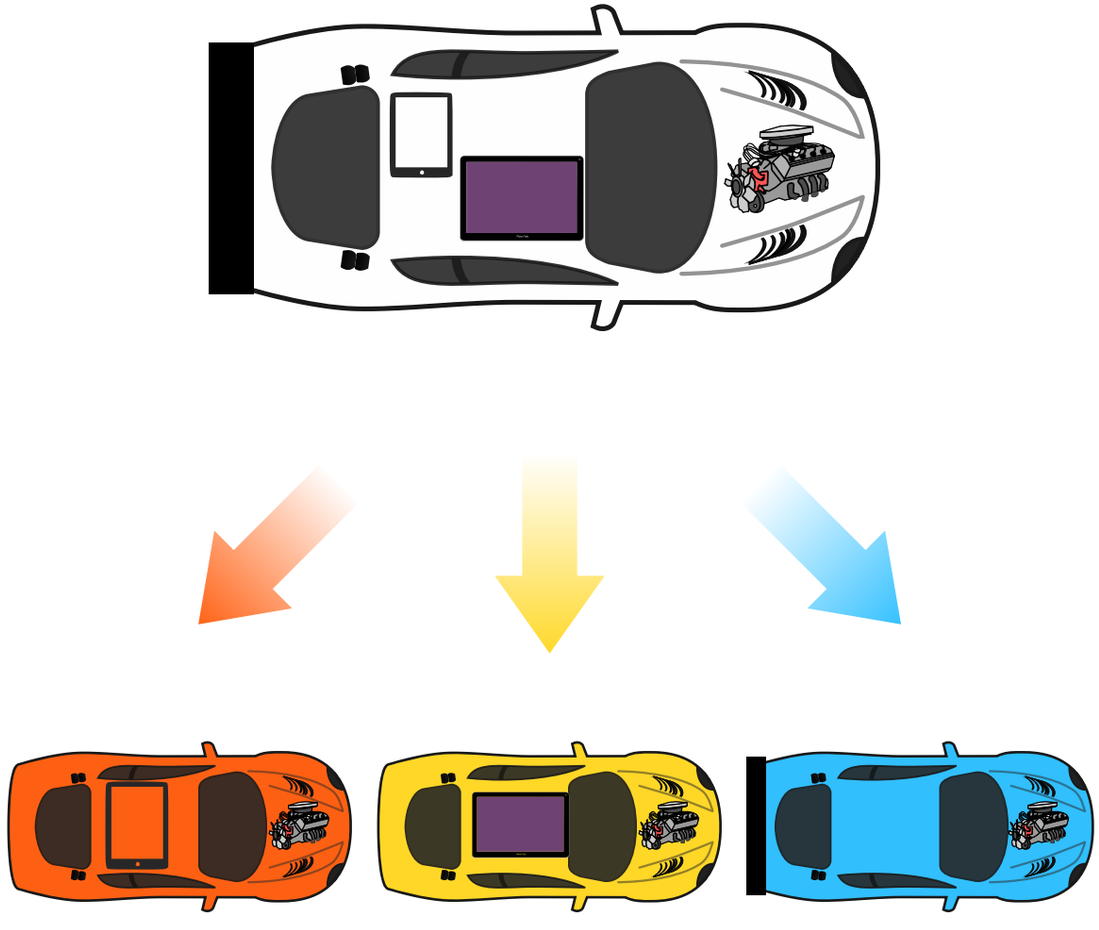
Single Product



Domain Engineering

Development *for* Reuse

Product Family



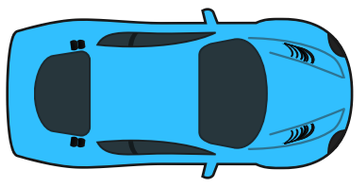
Why Domain Engineering?

Why Domain Engineering?

…because

opportunistic reuse *does not scale*

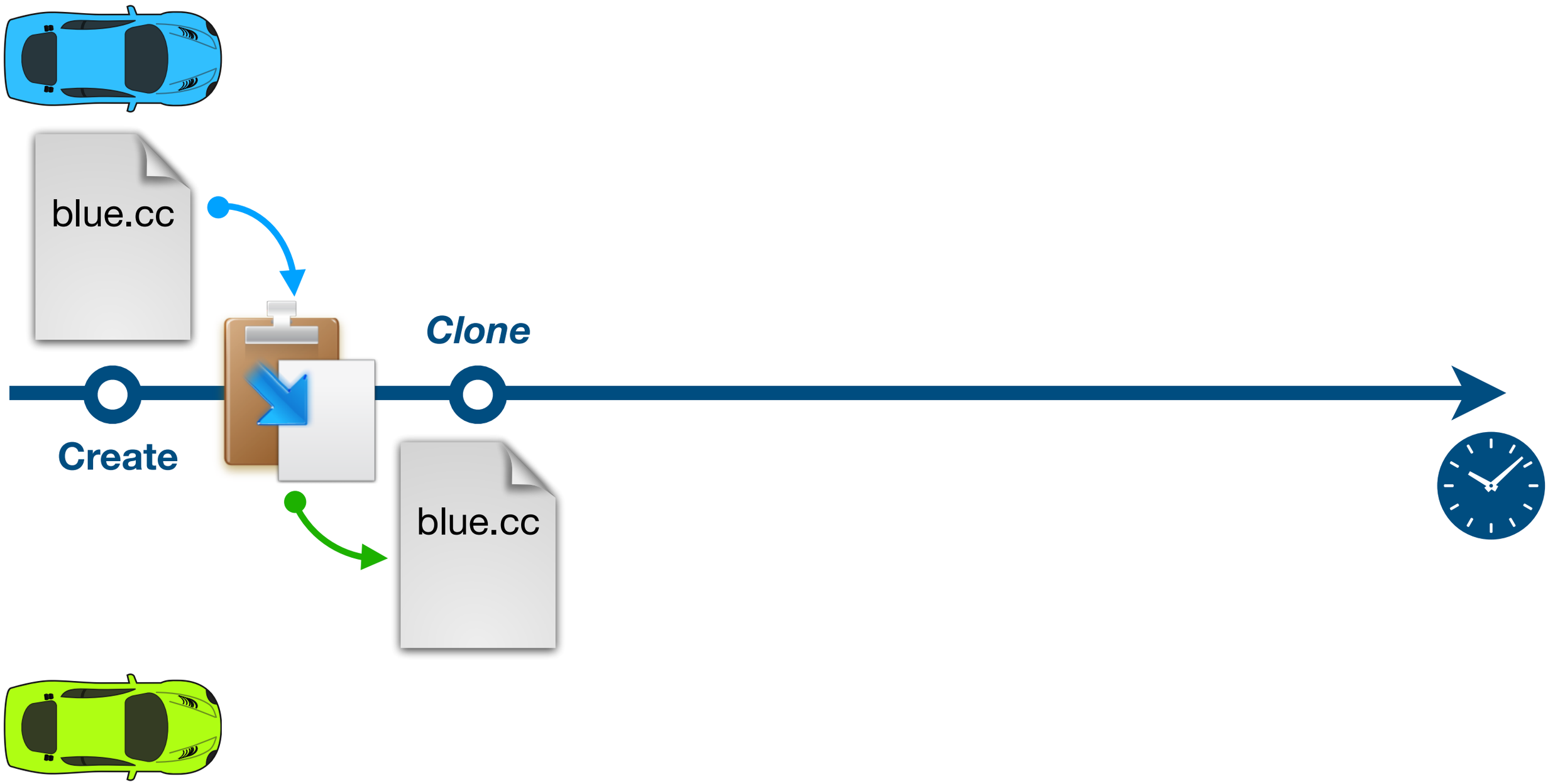
Clone and Own



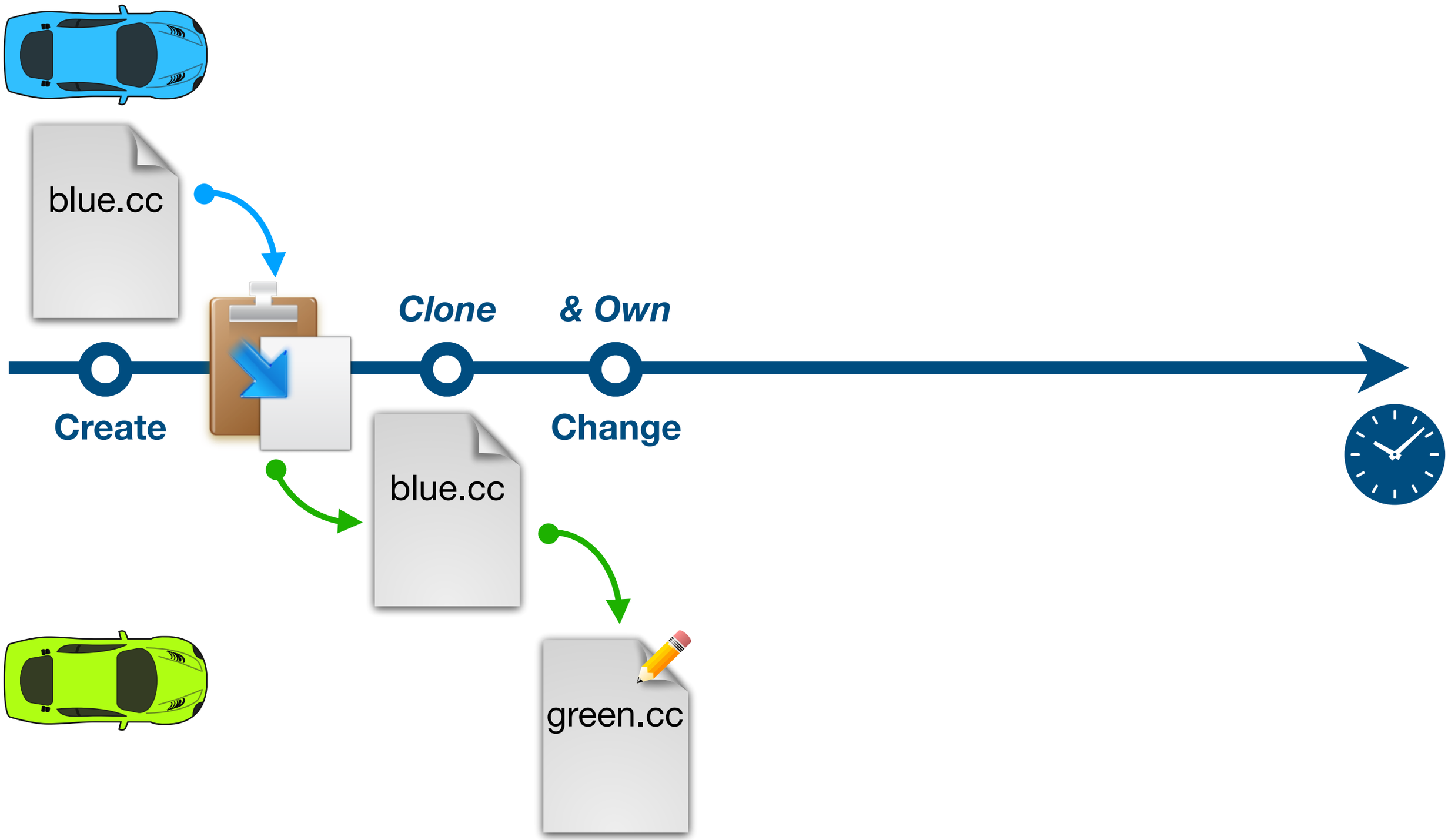




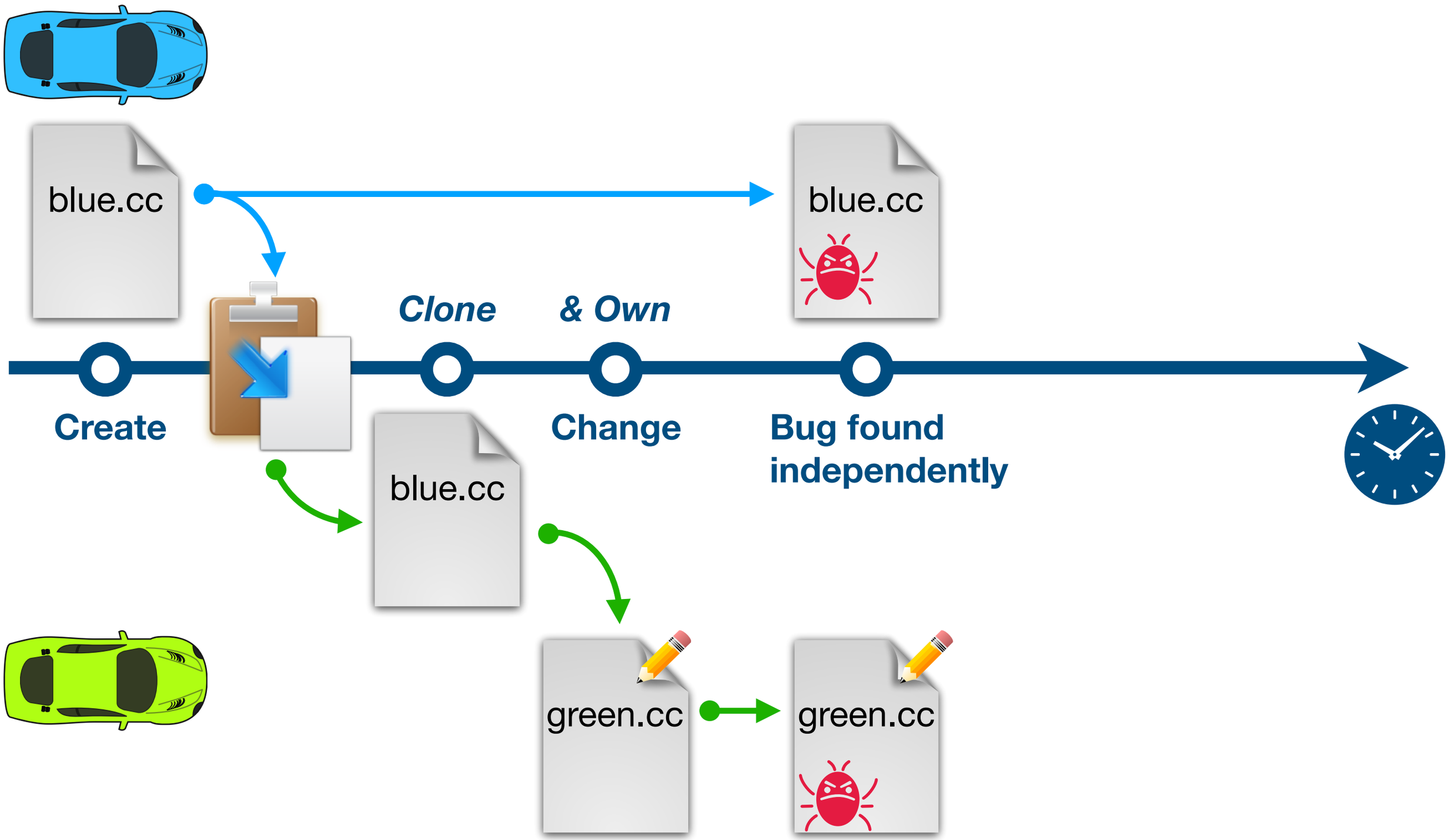
Clone and Own



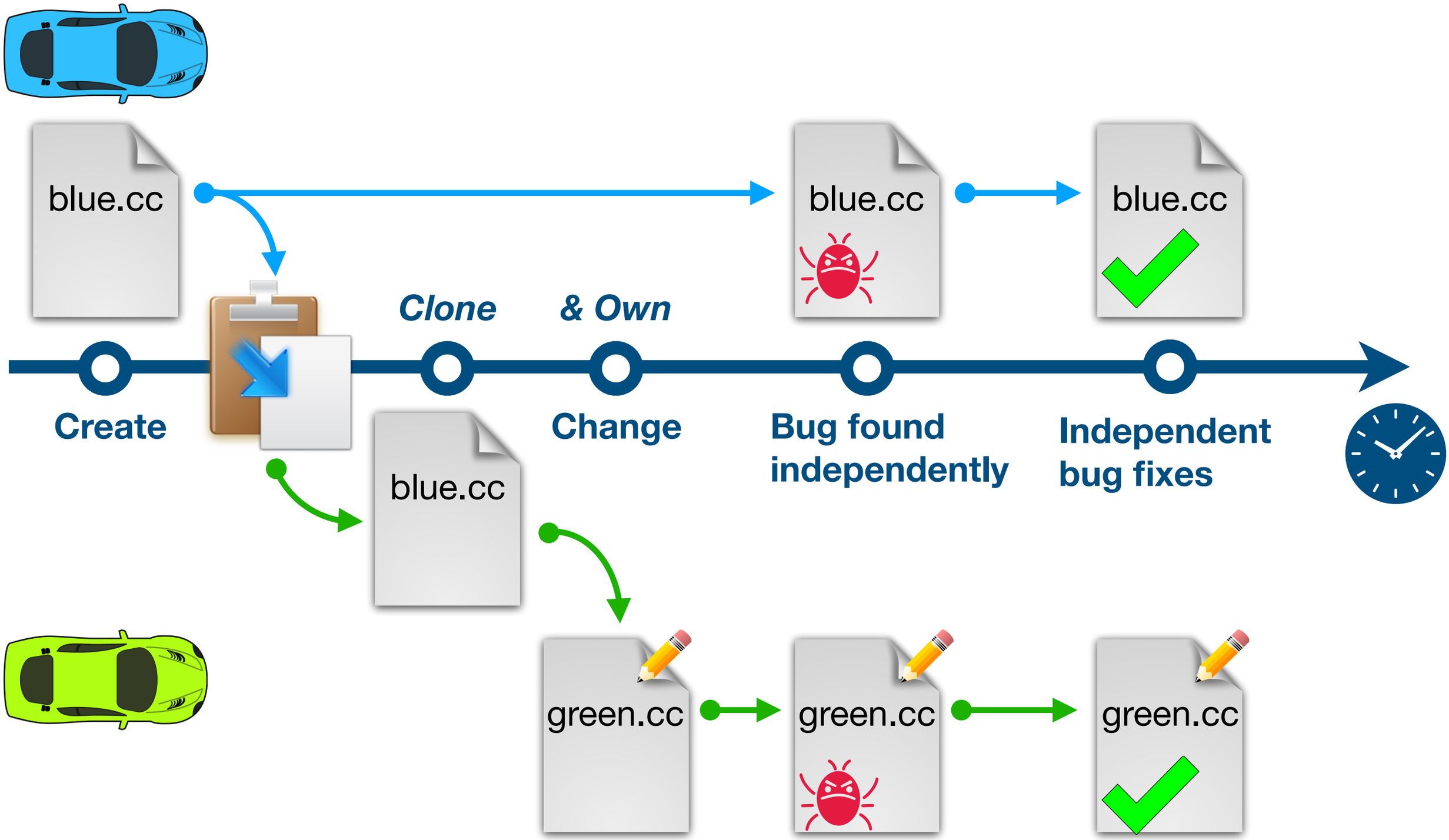
Clone and Own



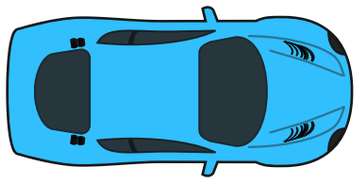
Clone and Own

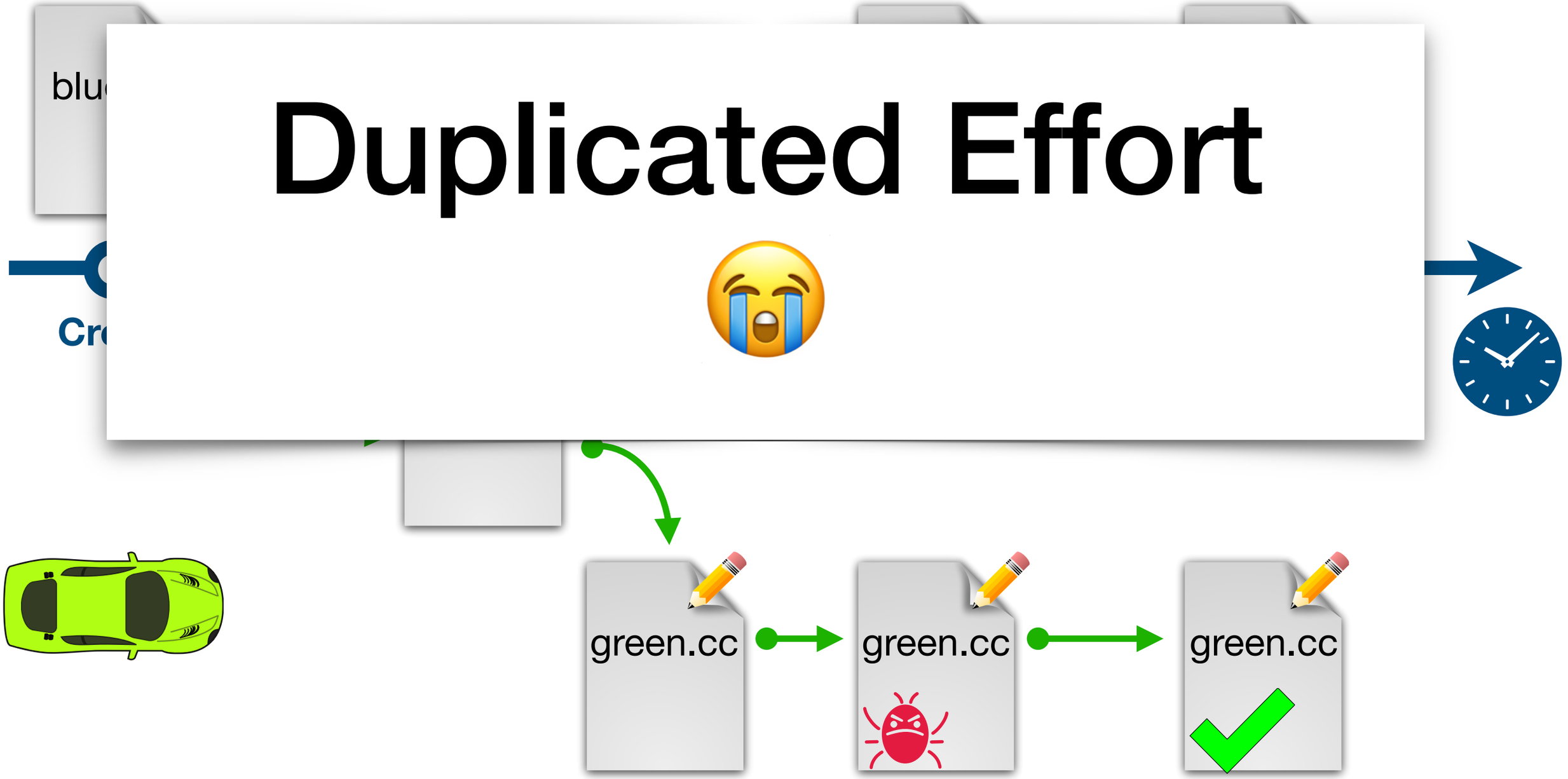


Clone and Own



Clone and Own





Duplicated Effort

😭

Cloning as Opportunistic Reuse

Dubinsky et al., Exploratory Study of Cloning in Industrial SPLs, CSMR 2013

+ **Easy**, no special tooling required

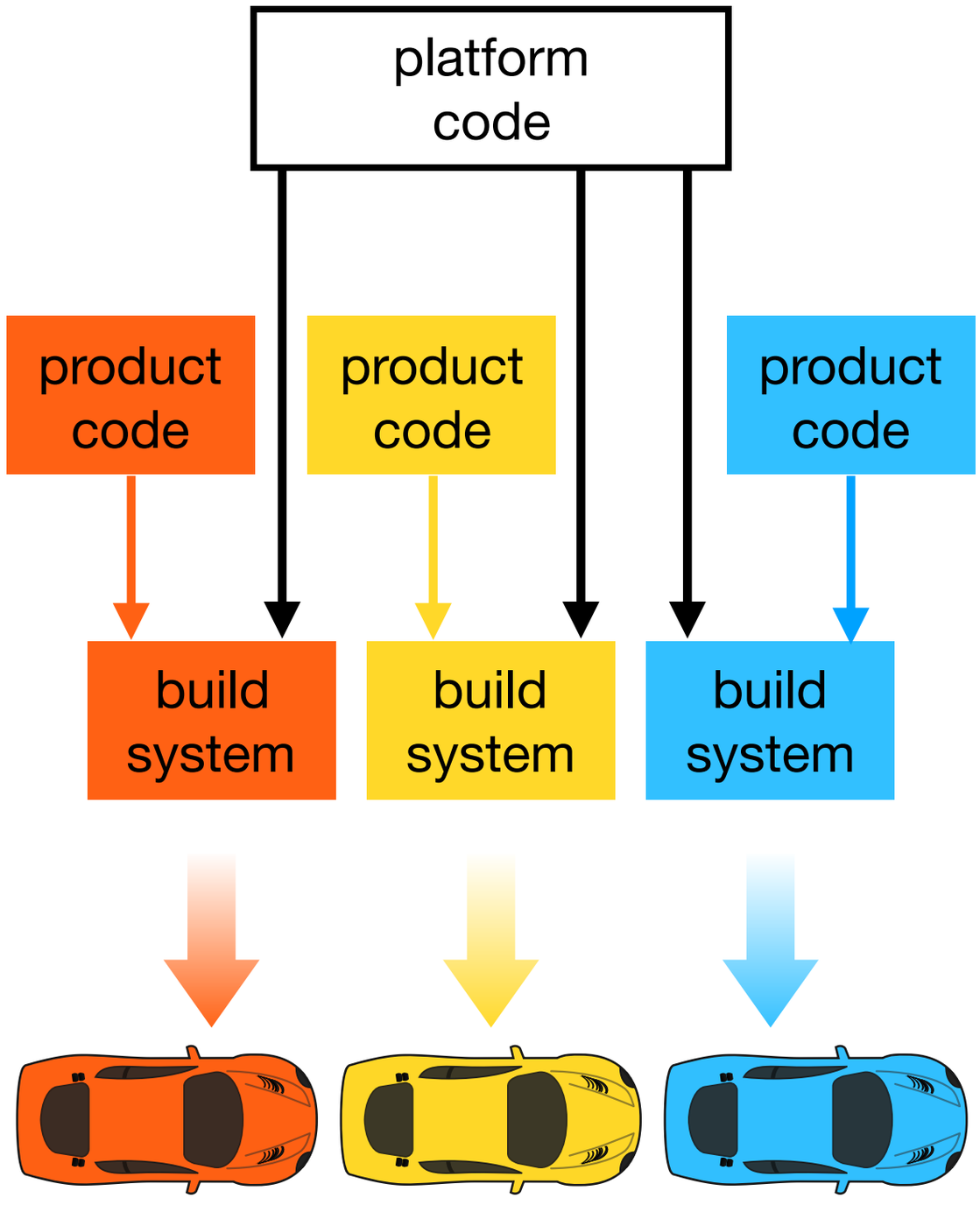
+ **Quickly** available functionality

- **No** sharing (fixes & features)

- Maintain **yourself** (test, debug, change)

- **Product specific** code grows

- Platform code **diminishes and degrades**



Successful Reuse

Proactive

Planned

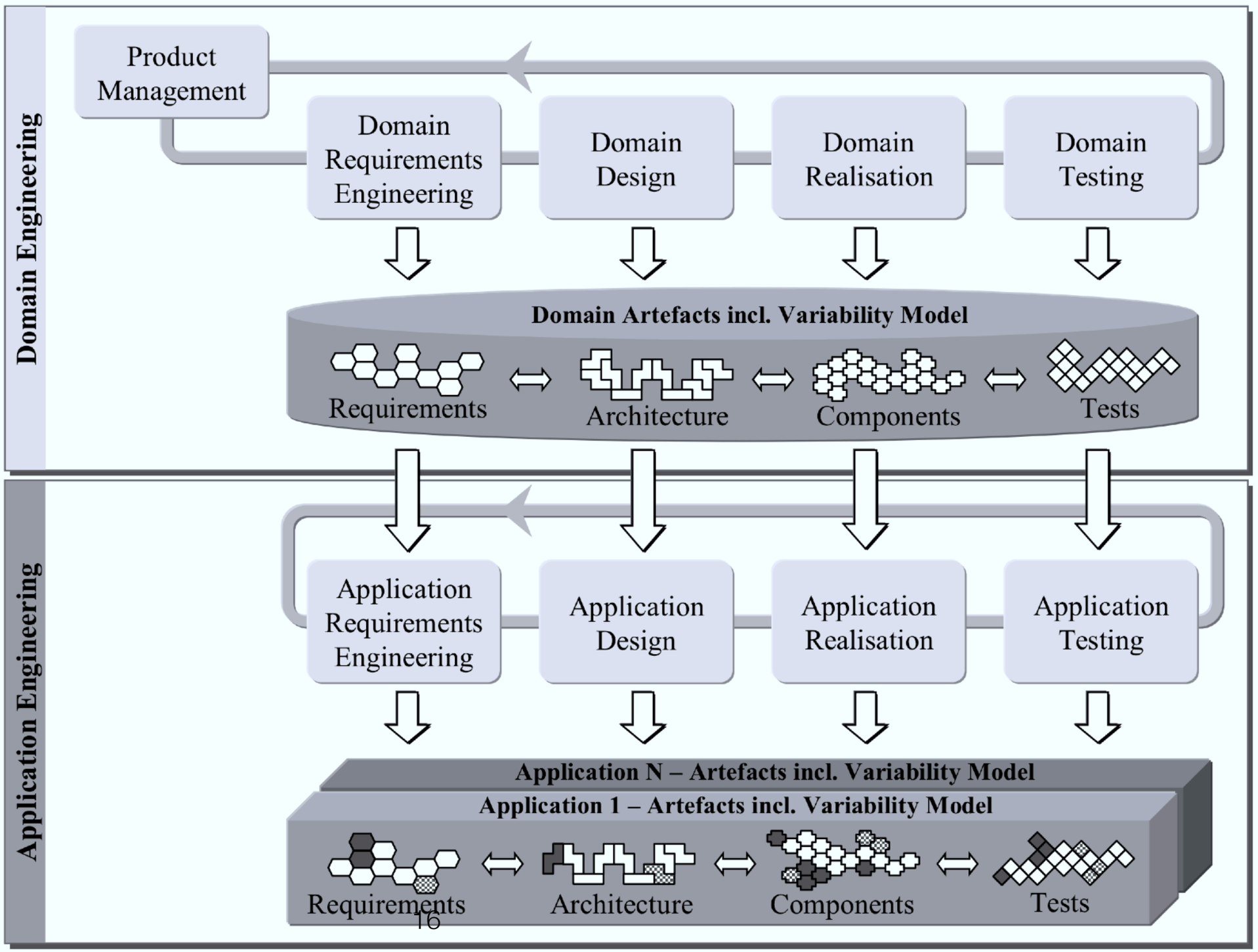
Managed

Domain vs Application Engineering

Pohl et al., Software Product Line Engineering, Springer Science, 2005

product independent

product specific

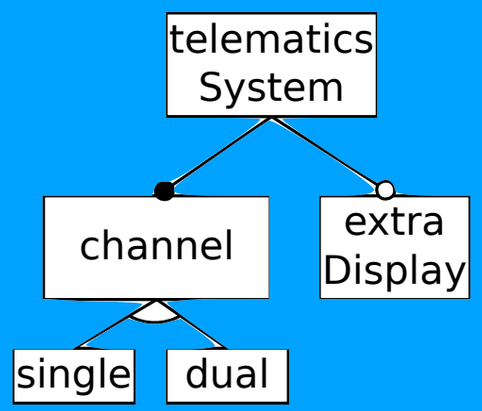


features

variation points

Problem Space

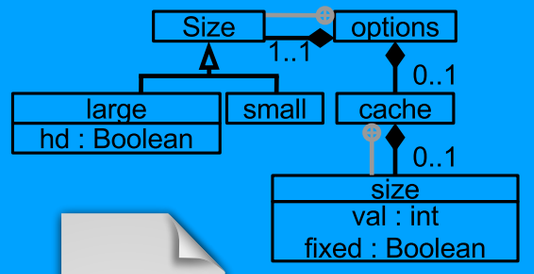
domain-specific abstractions



Mapping

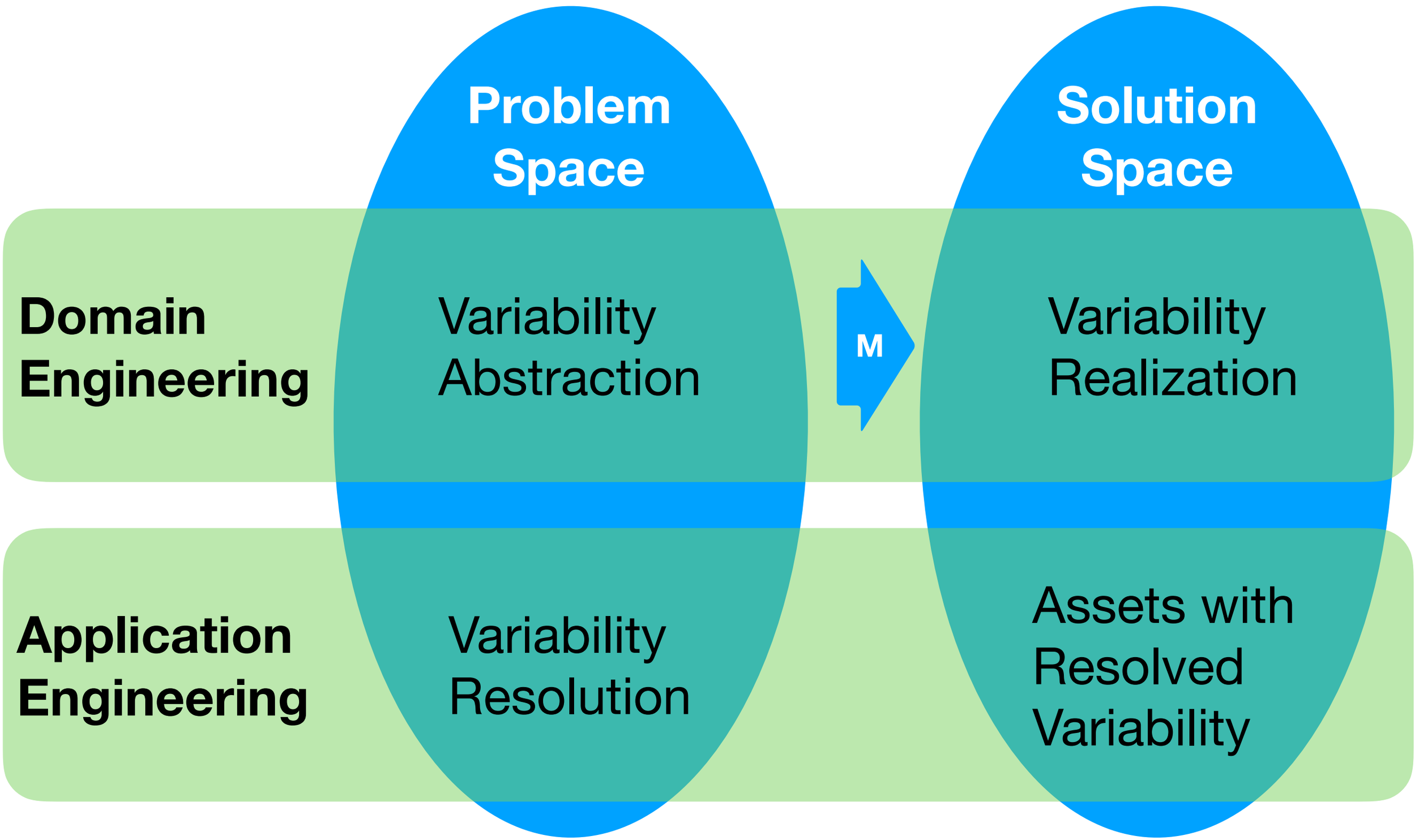
Solution Space

implementation oriented abstractions

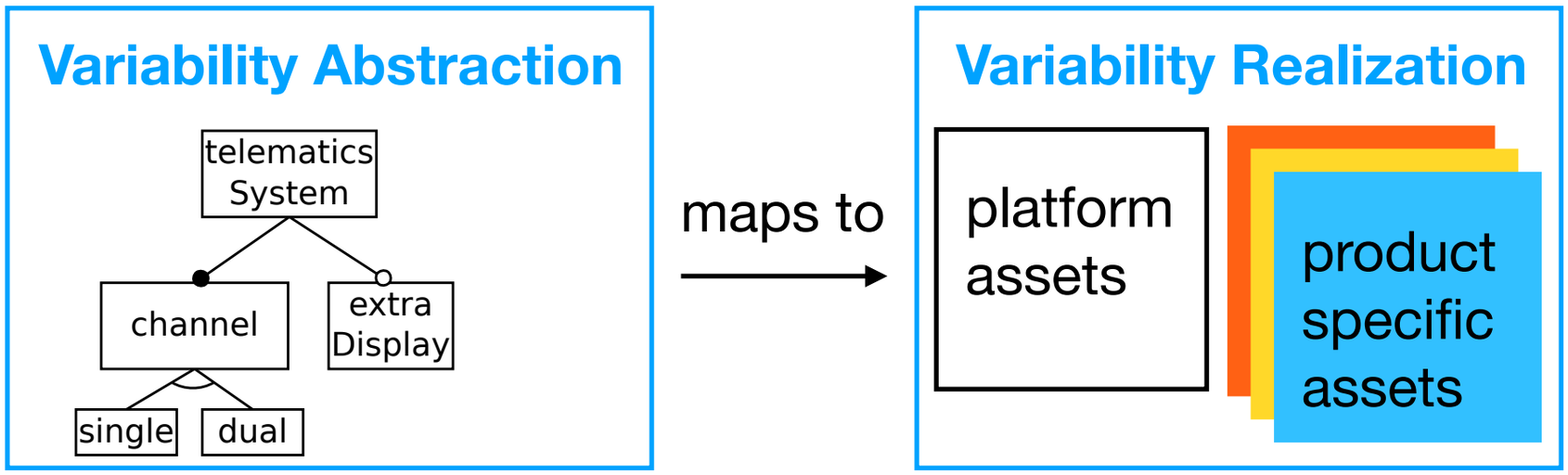




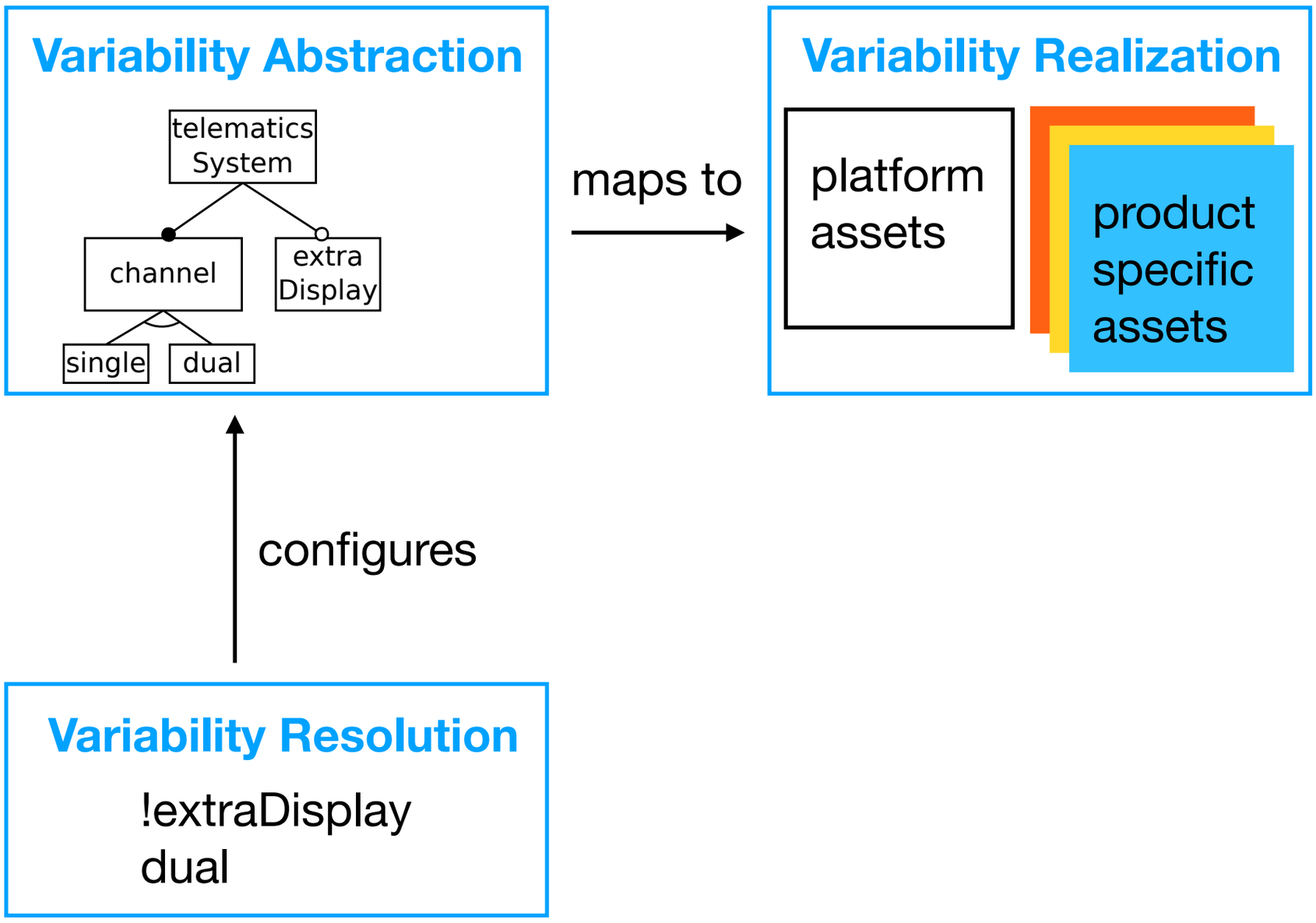
Software Product Line



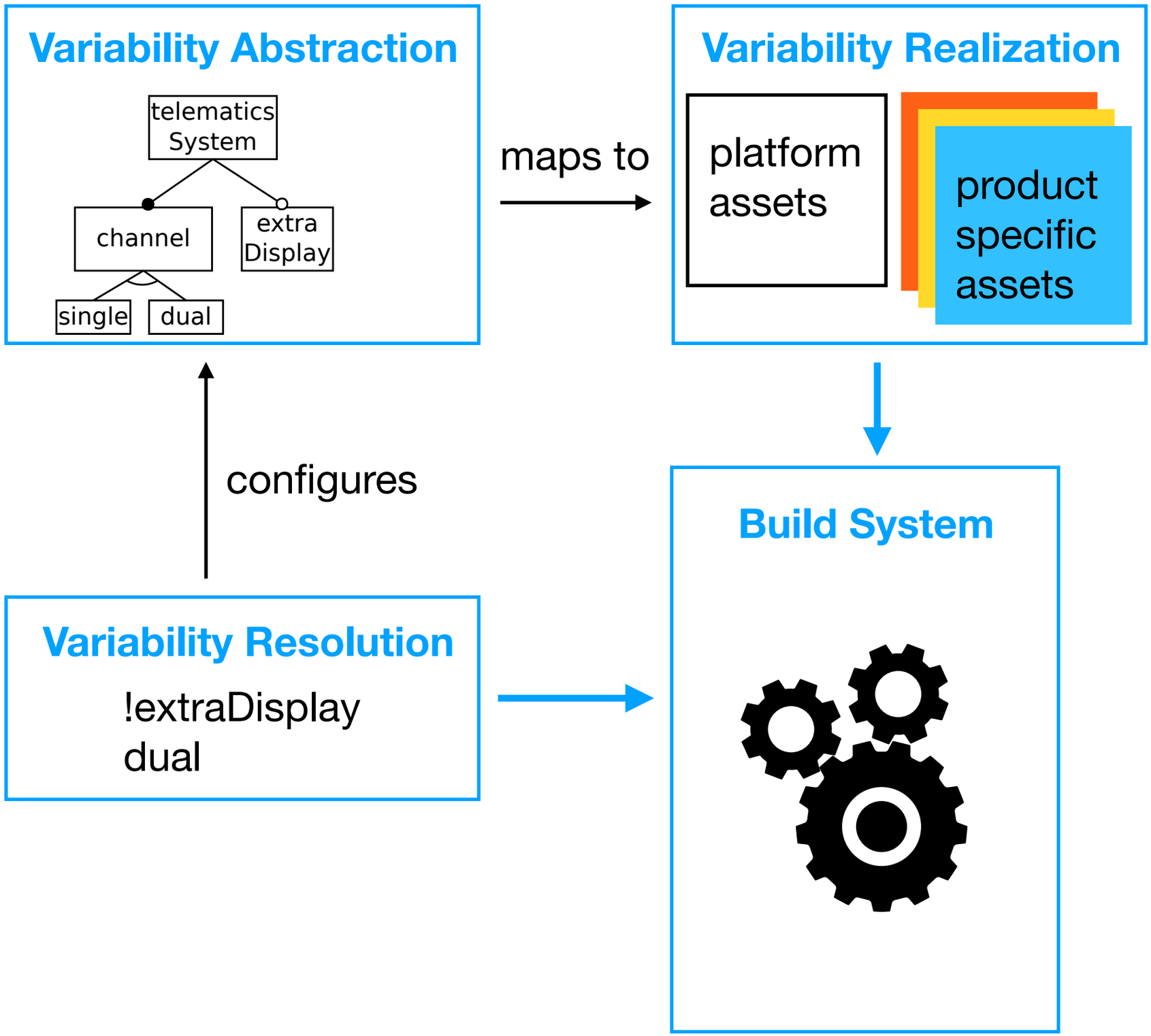
Product Line Architecture Overview



Product Line Architecture Overview

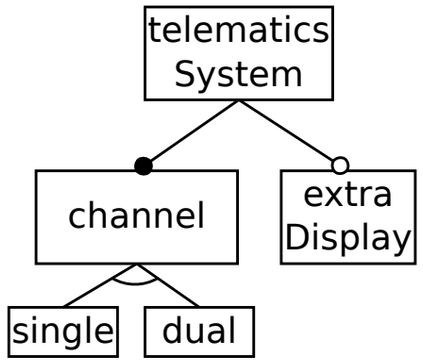


Product Line Architecture Overview



Product Line Architecture Overview

Variability Abstraction



maps to

Variability Realization

platform assets

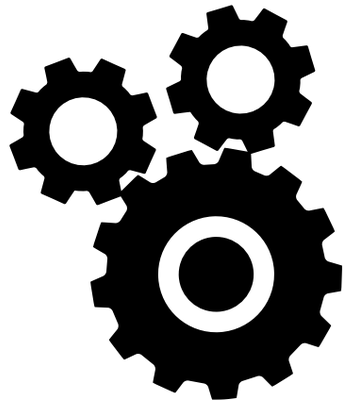
product specific assets

configures

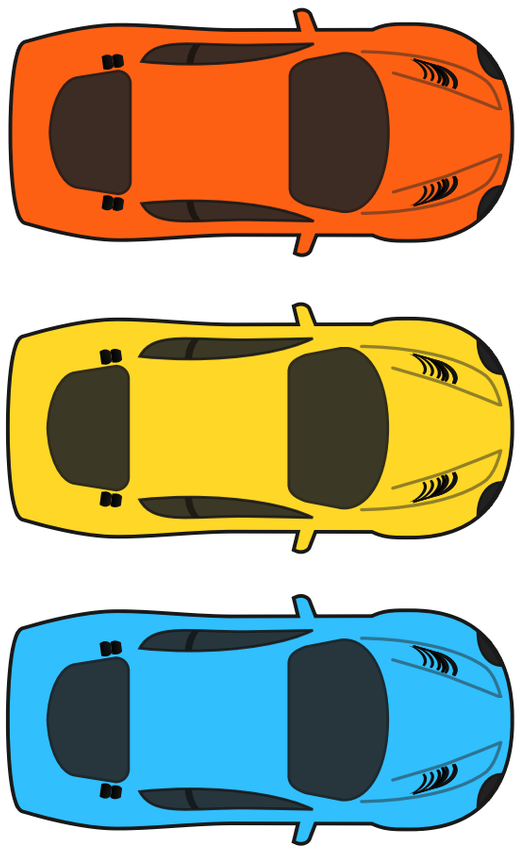
Variability Resolution

!extraDisplay
dual

Build System

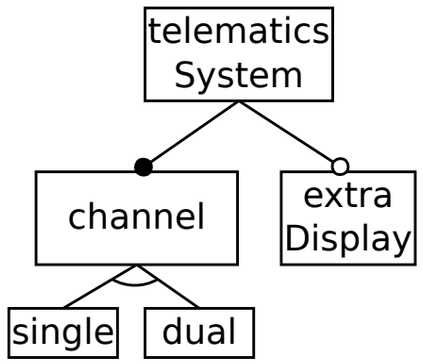


Assets with Resolved Variability



Implementation Technologies

Variability Abstraction



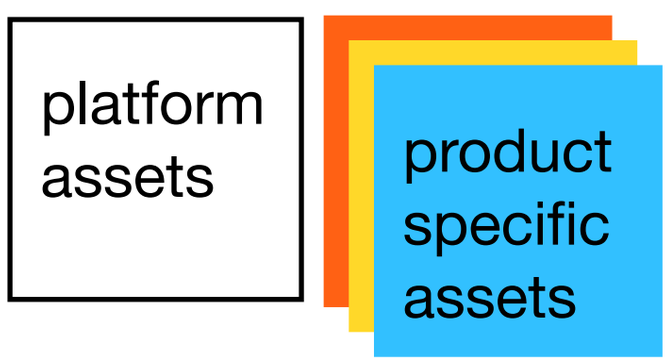
- **Feature models**
- Domain Specific Languages
- *none*

Variability Resolution

!extraDisplay
dual

- **Feature model configuration, constraints**
- Domain specific model
- XML, JSON, custom text format, …

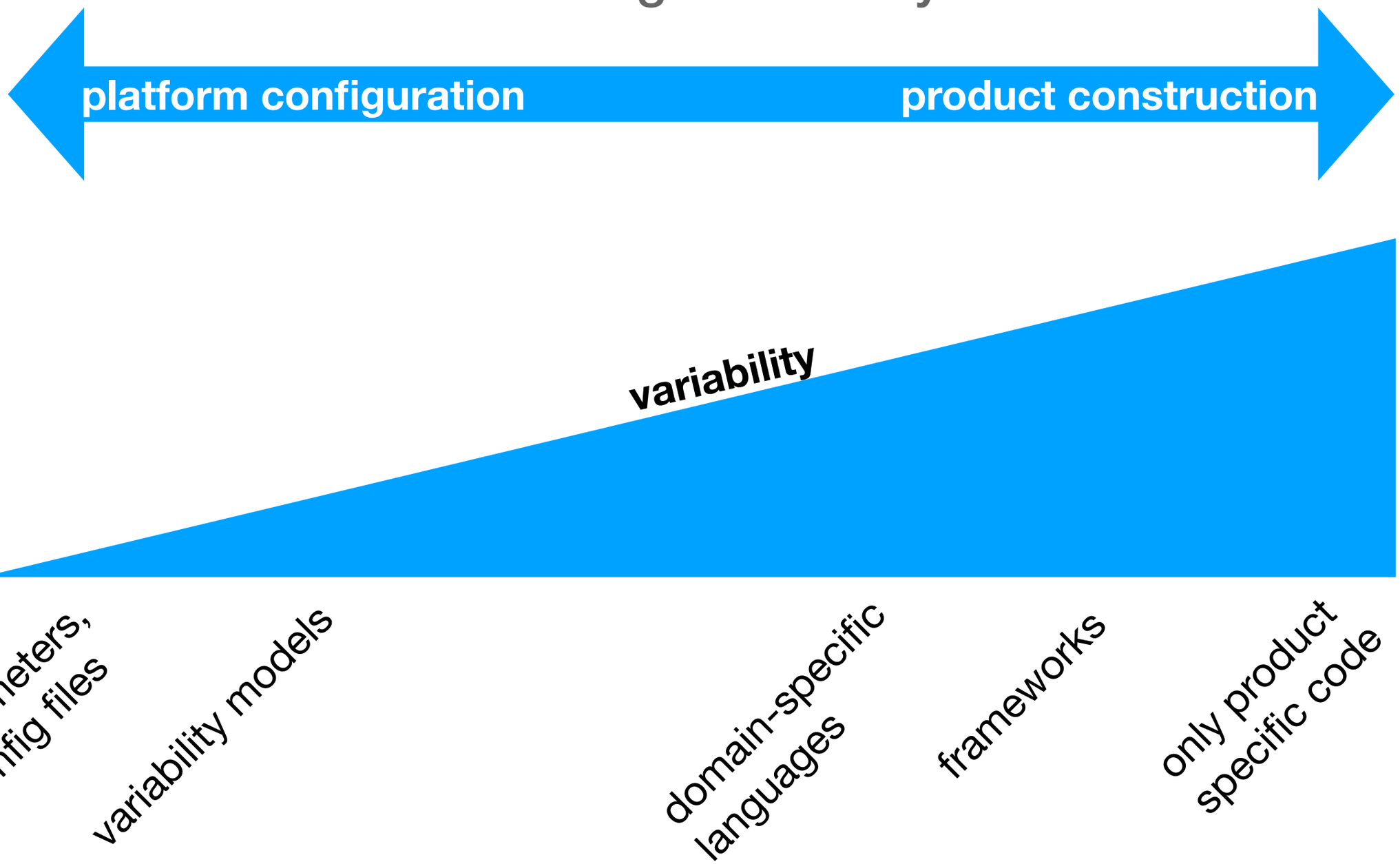
Variability Realization



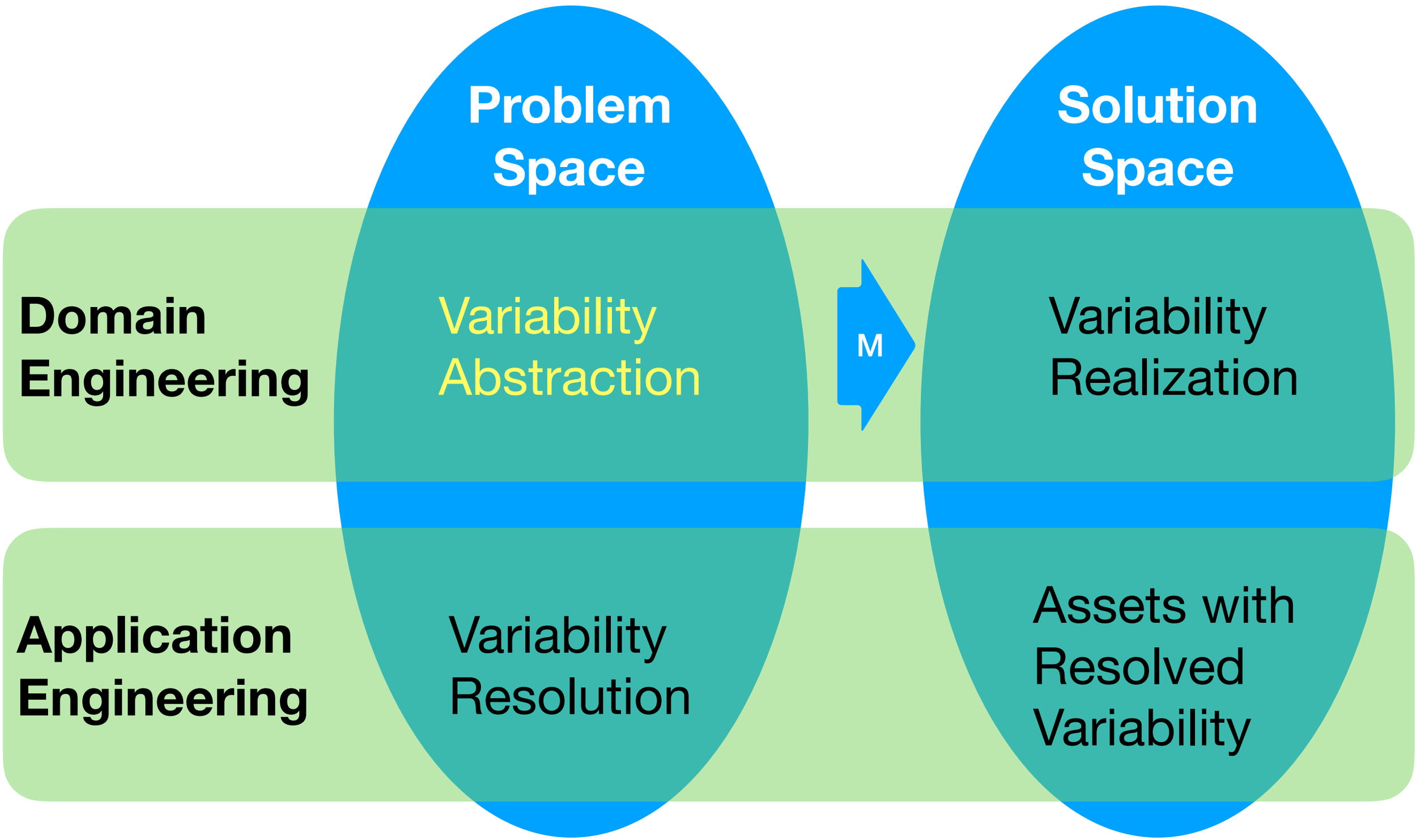
- Code (with variability techniques)
- Code generators
- Model transformers
- Parts may use DSLs

Spectrum of Variability Architectures

Stay as Close to the Left as Possible
Exploit Commonality
Manage Variability



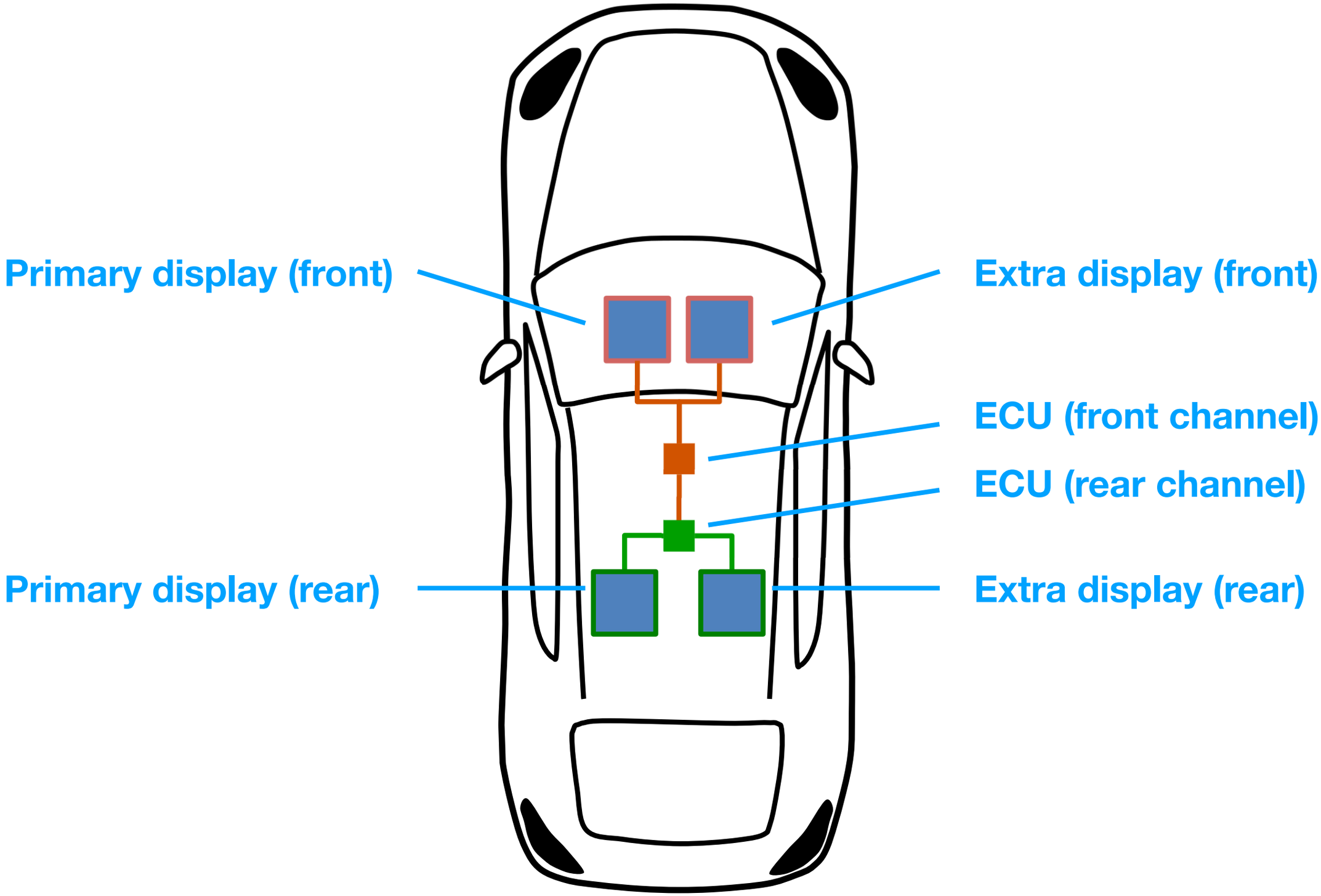
Software Product Line



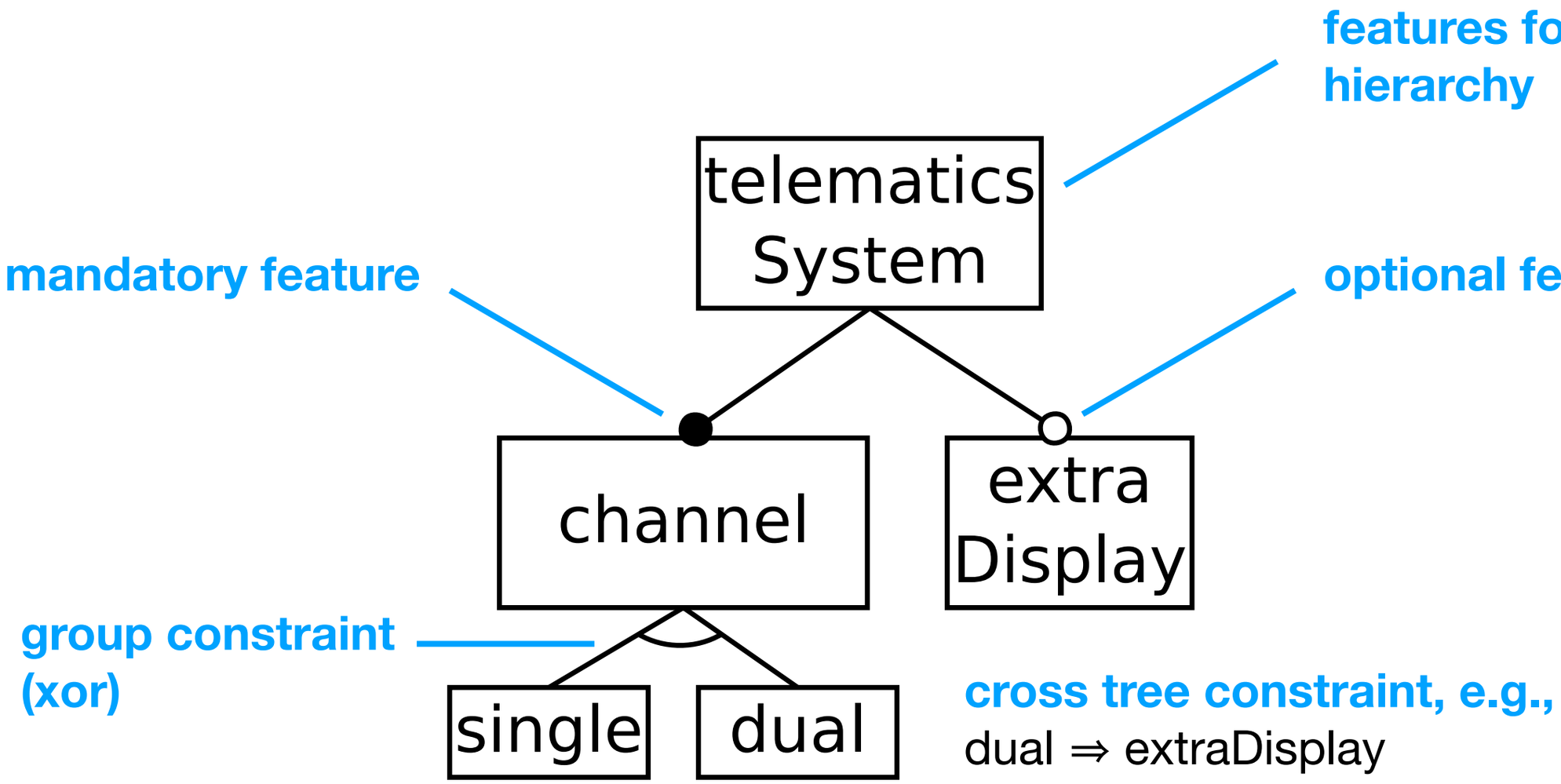
Example: Telematics System





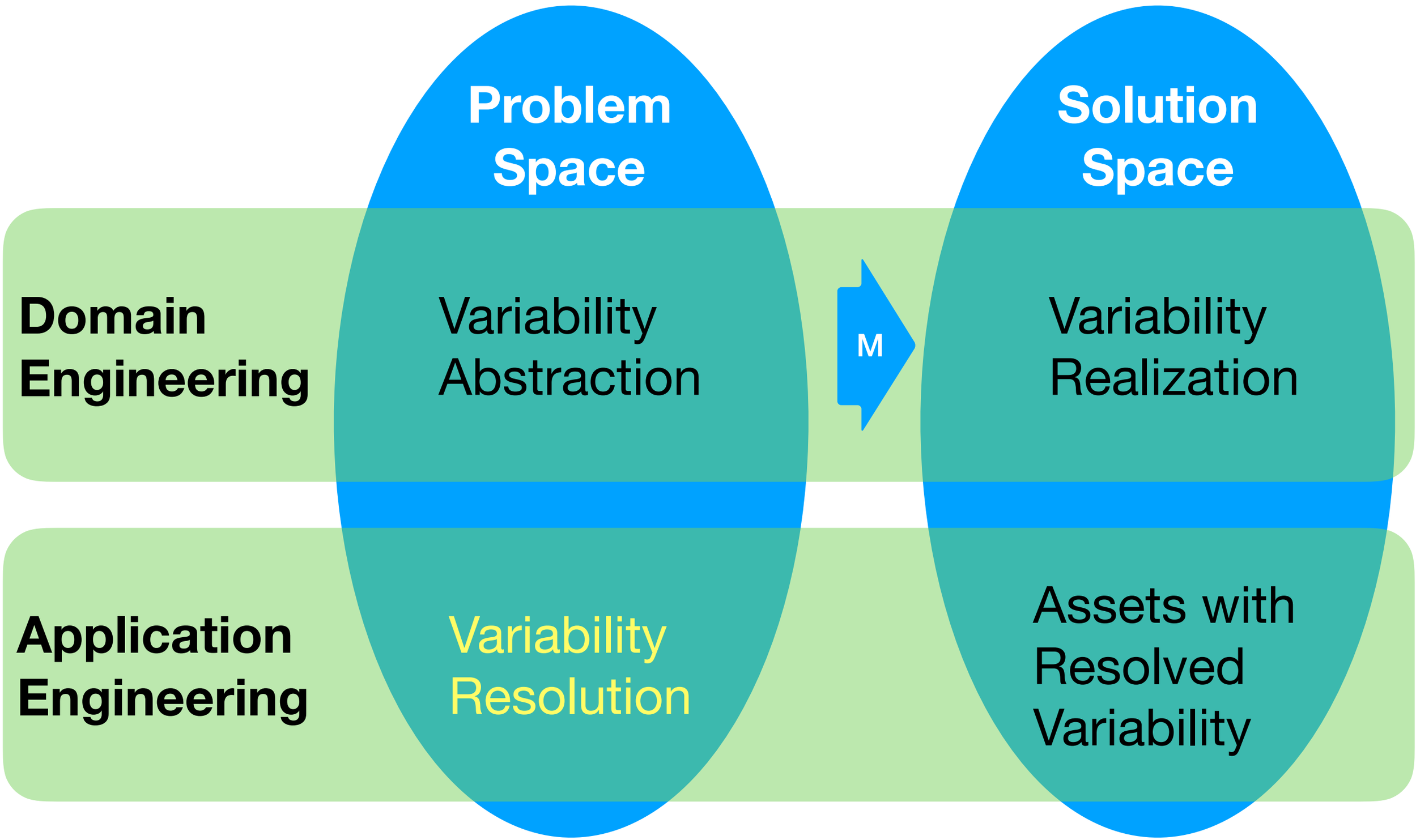

Primary display (front)
Extra display (front)
ECU (front channel)
ECU (rear channel)
Primary display (rear)
Extra display (rear)

Feature Model

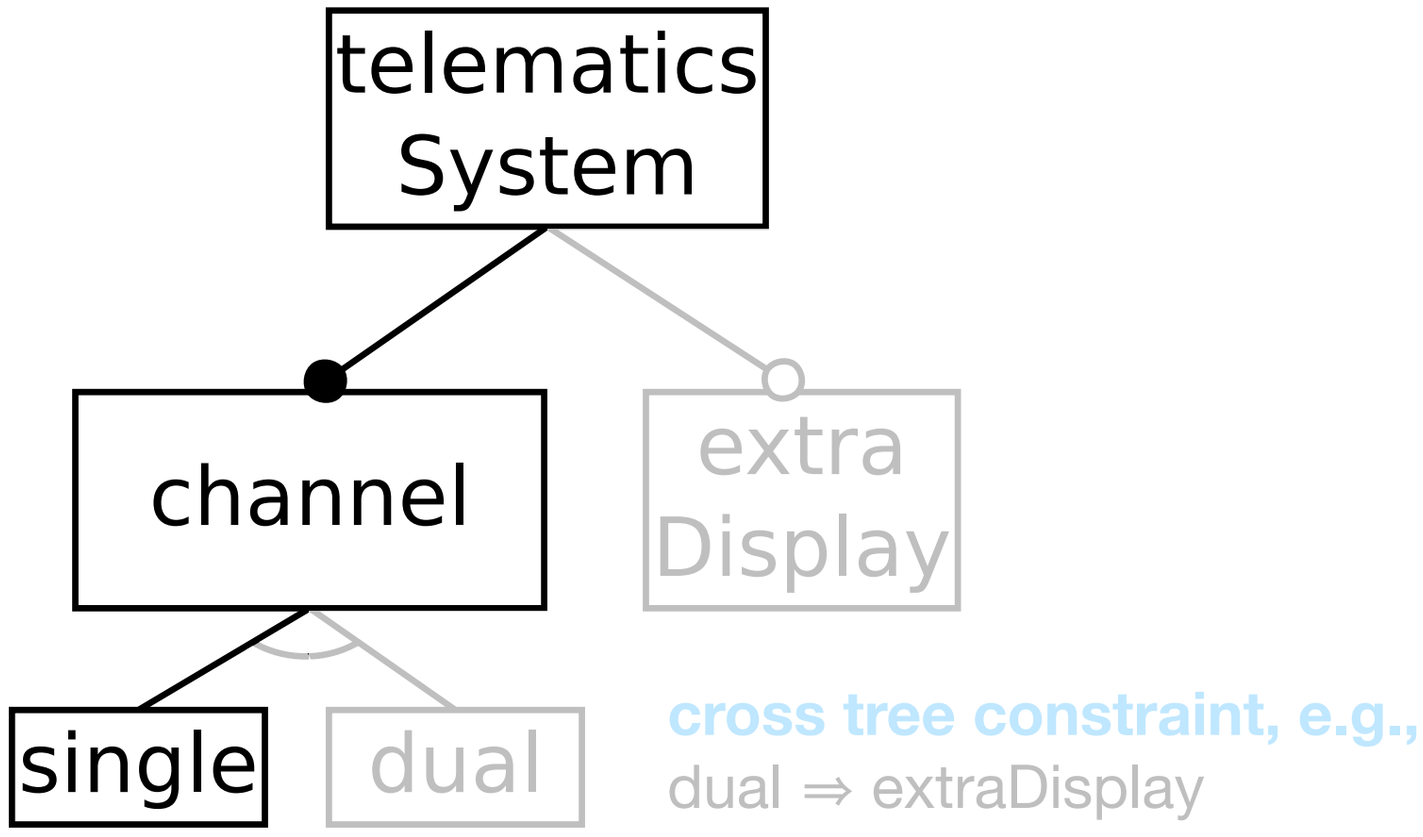


single kind of relationship: *subfeature*
meaning: *implication*

Software Product Line



Feature Configuration

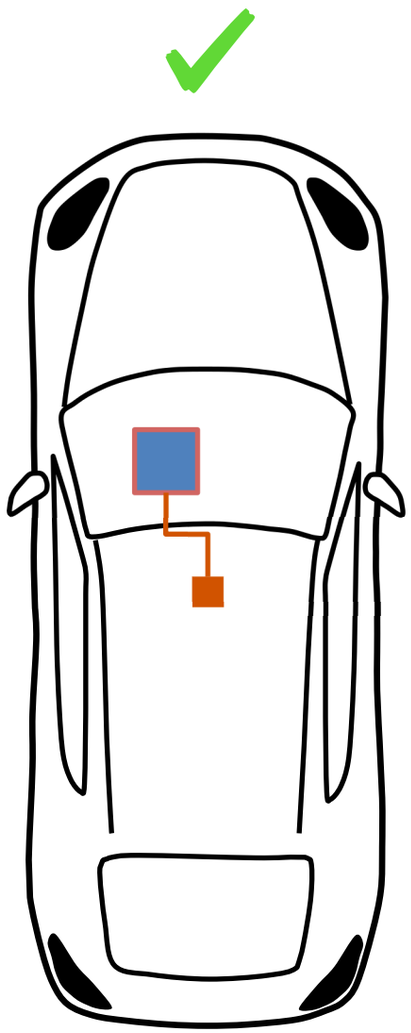


cross tree constraint, e.g.,
dual ⇒ extraDisplay

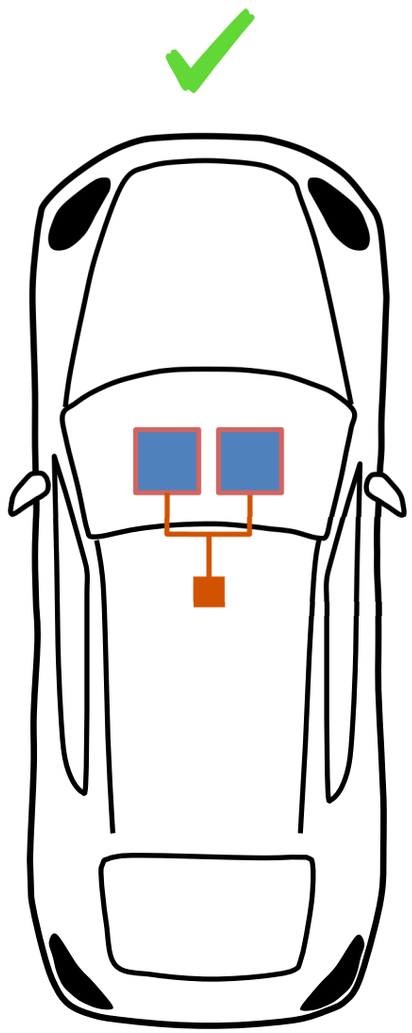
{ telematicsSystem,
channel, **single** }

Configuration Semantics of an FM

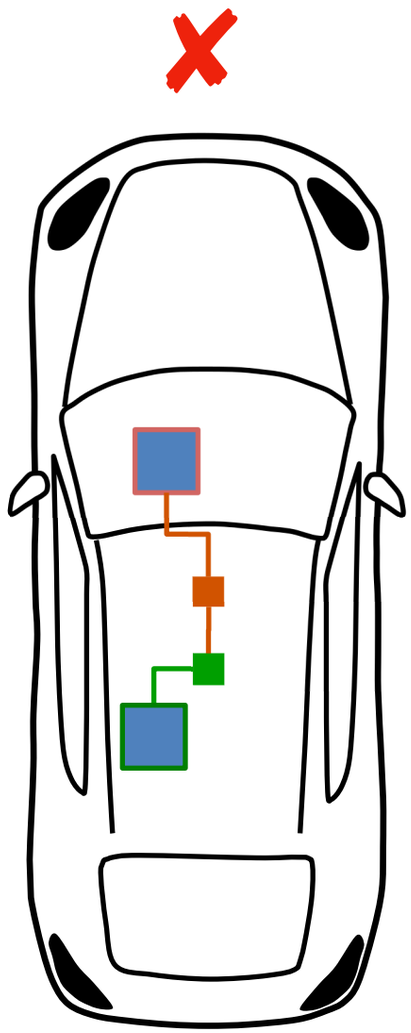
Set of Configurations



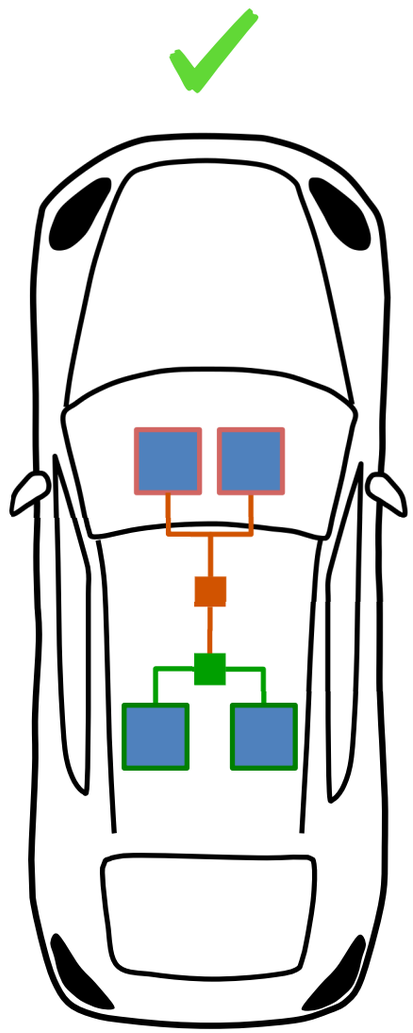
{ telematicsSystem, channel, **single** }



{ telematicsSystem, channel, **single**, **extraDisplay** }



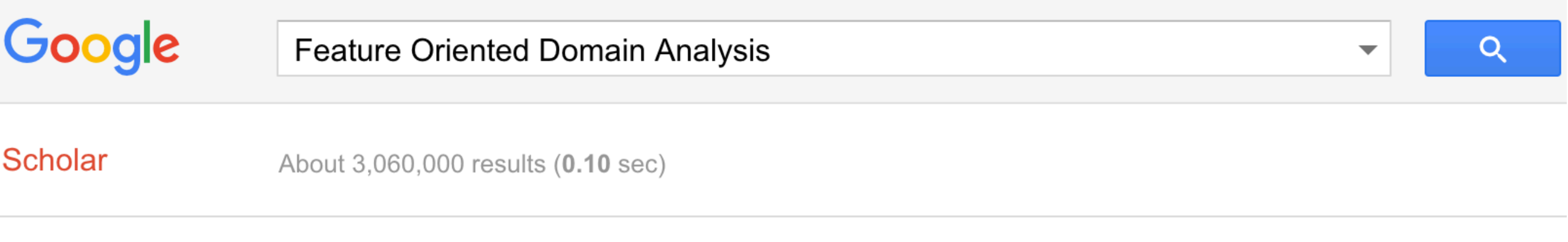
{ telematicsSystem, channel, **dual** }



{ telematicsSystem, channel, **dual**, **extraDisplay** }

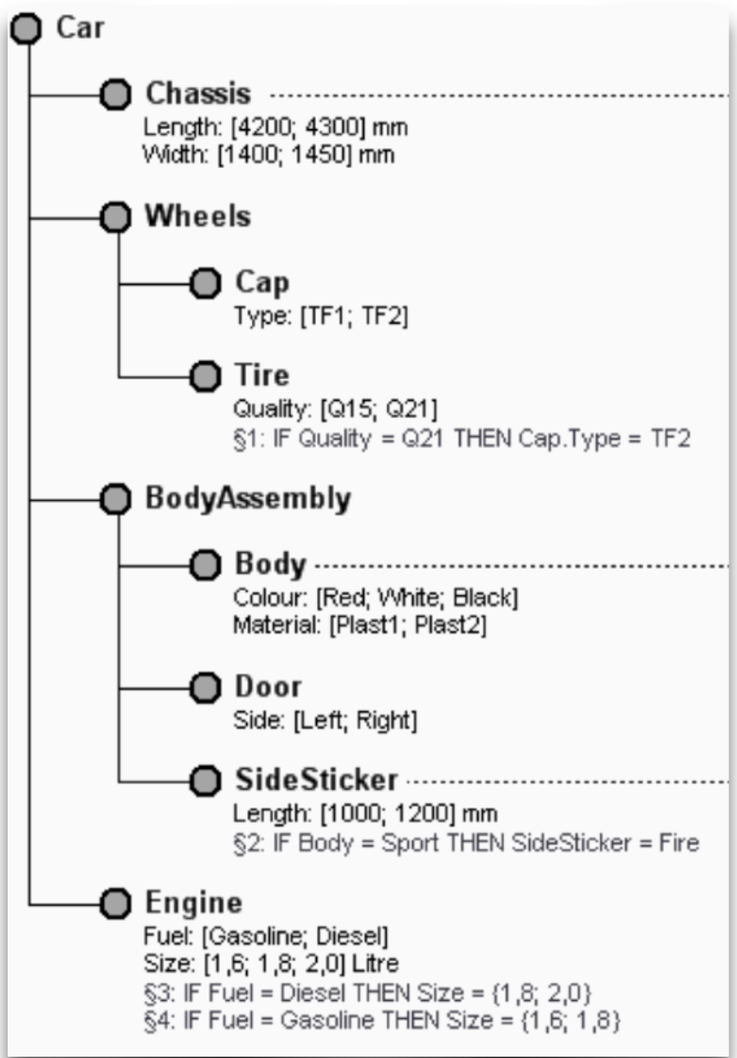
Feature Modeling and FODA

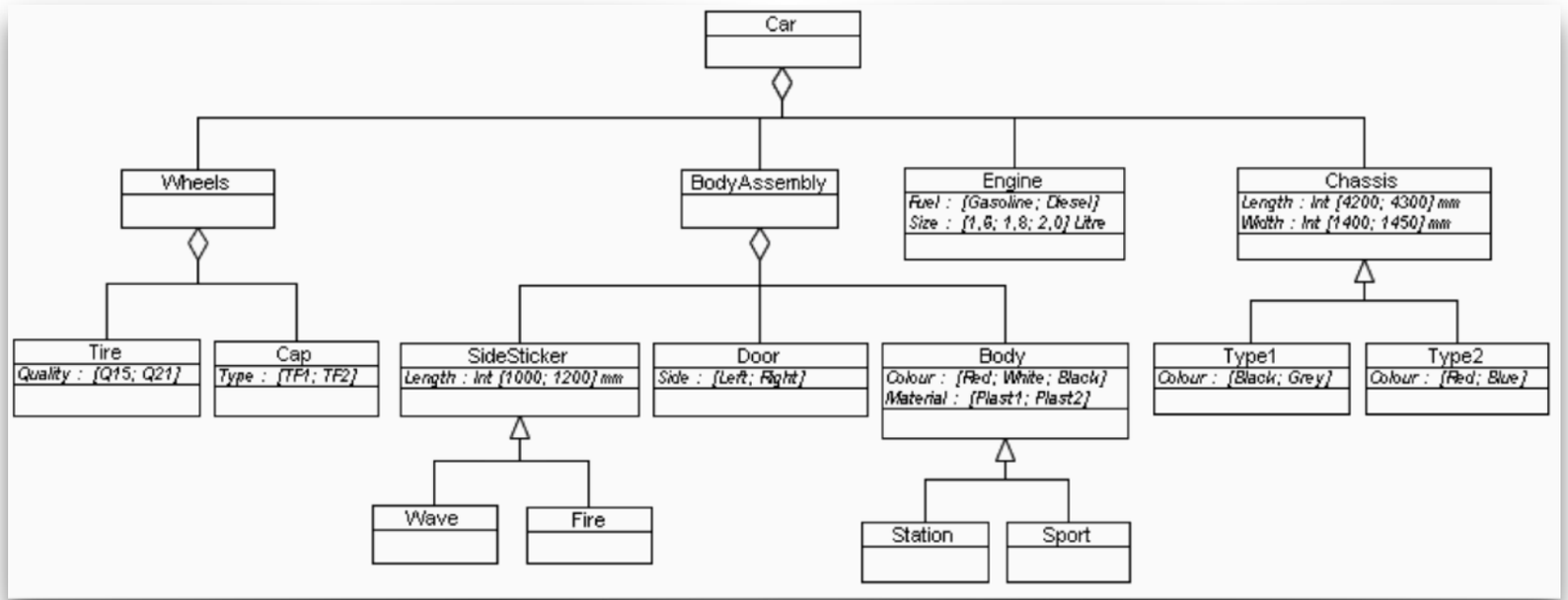
- FODA succeeds for its **simplicity**
- Probably best intro in Czarnecki's *Generative Programming* (Ch. 4)
- 4700+ citations, **never formally published**



Feature Models vs Class Models

A Feature Model in *Product Variant Master* Notation (Hvam)





Haug et al., Creating a documentation system to support the development and maintenance of product configuration systems, WSEAS 2007

Feature Models vs Class Models

Concepts	**Few and simple**: feature, subfeature, group, constraint	**Many and complex**: class, generalization, composition, association, redefinition, refinement, property, multiplicity, package, data type, primitive type, enumeration, …
Use	**Variation** of *user-relevant* characteristics of product variants	**Concepts** representing more detailed aspects of products; *product line architectures*
Semantics	**Configuration** - selections from *predefined choices* within a fixed tree structure	**Instantiation** - making *new* structures that conform to predefined types, and *connecting* them via links

Bąk et al., Clafer: Unifying Class and Feature Modeling, SOSYM 2014

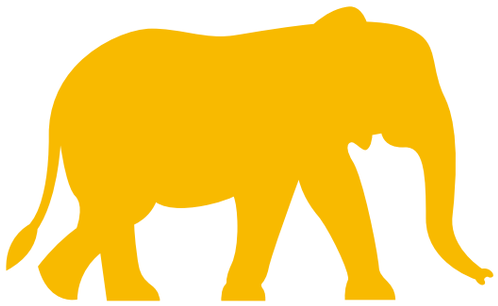
How to Build Feature Models?

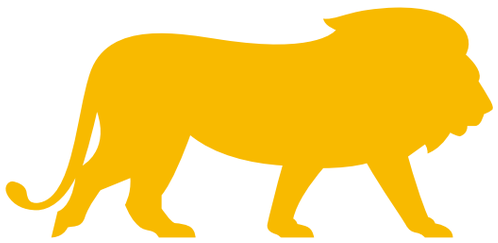
Bottom Up - Incremental Adoption

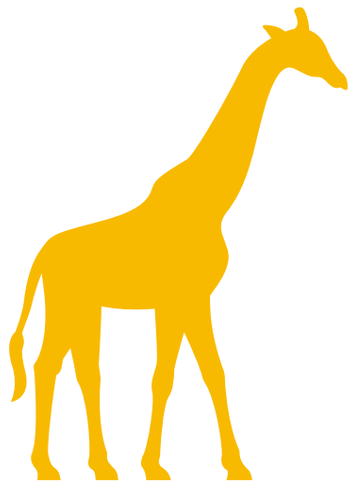
- Identify **cloned** code/functionality
- Find the **patches** that describe differences
- Diffs → **variation points**
- Aggregate variation points into hierarchical **features**

Jepsen et al., Minimally Invasive Migration to Software Product Lines, SPLC 2007
Berger et al., A survey of variability modeling in industrial practice, VAMOS 2013

Variability Modeling in the Wild

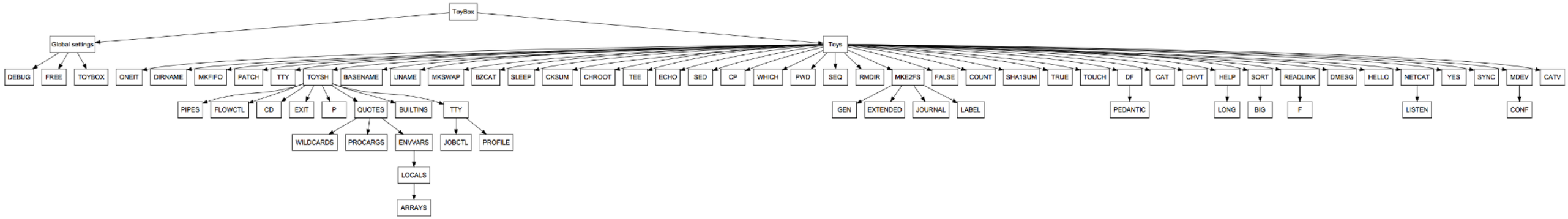






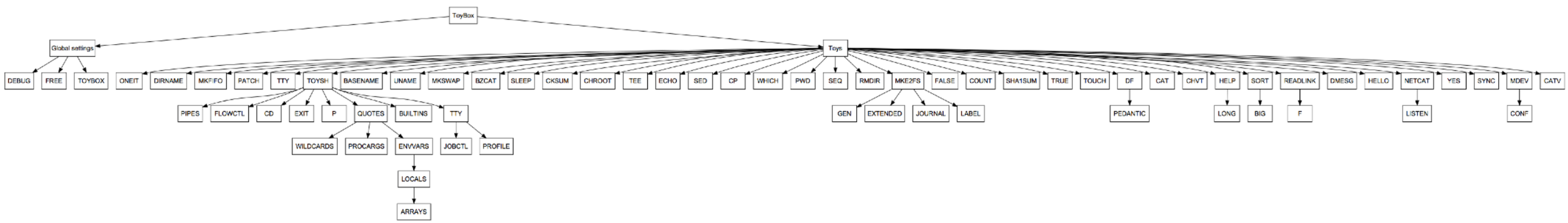
Healthy Wild Variability Model Club

ToyBox Project, 71 Features



Healthy Wild Variability Model Club

ToyBox Project, 71 Features



The Linux Kernel has 6-12k features, depending on how you count.

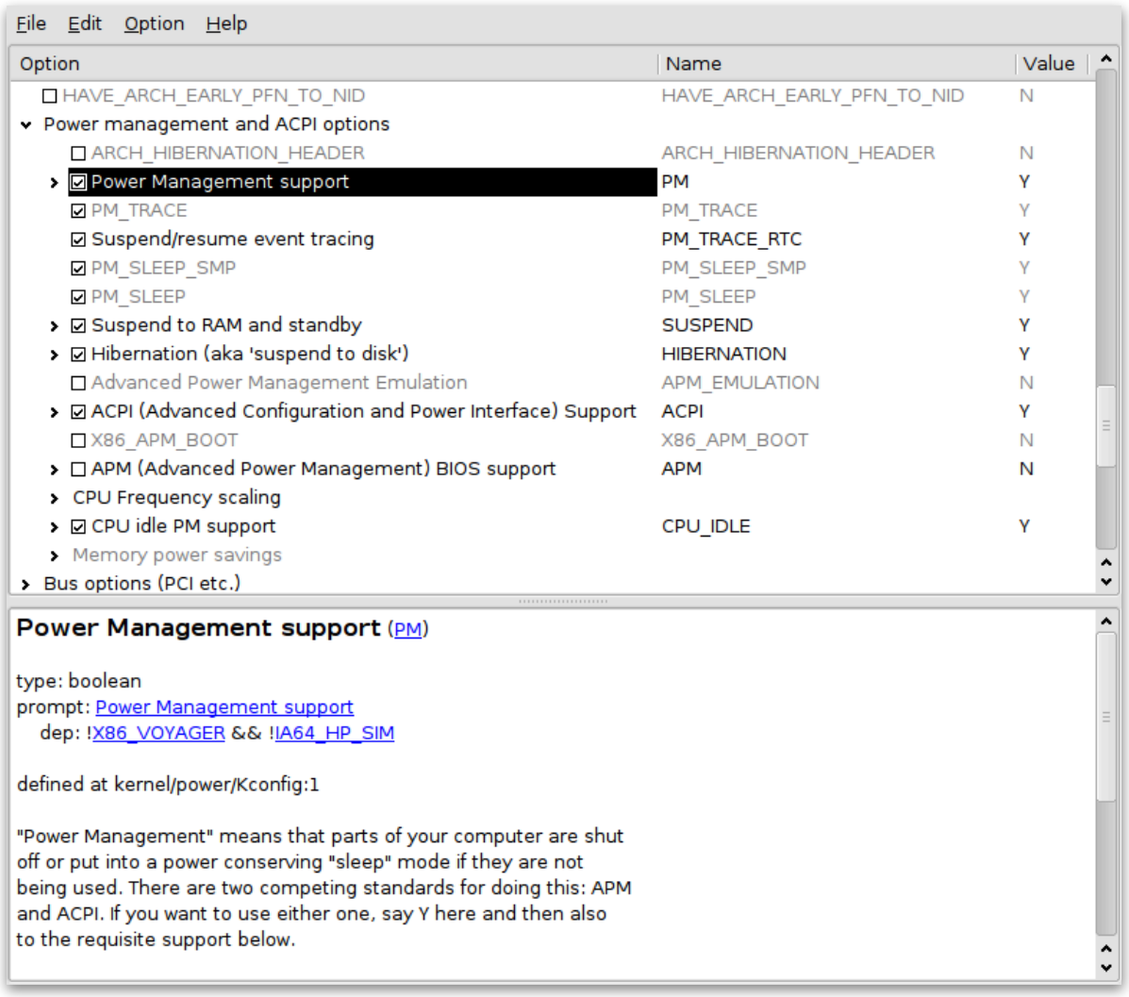
Max depth: 8. Most leaves are at 4! **Shallow**

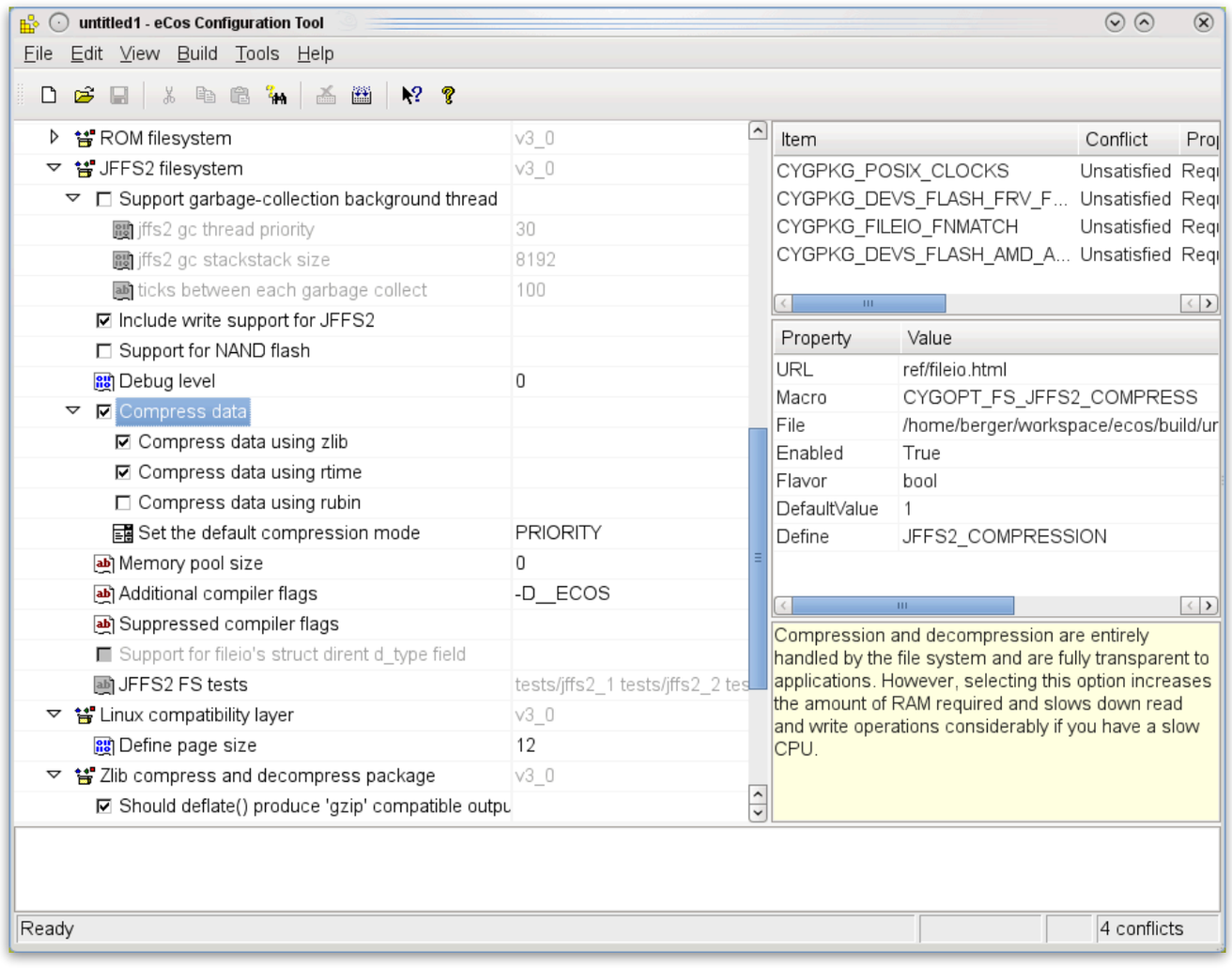
↓ this is the Linux Kernel model fit to the slide width ↓

Berger et al., A Study of Variability Models and Languages in the Systems Software Domain, TSE 2013

KConfig & CDL

FM with Attributes, Defaults, Constraint Propagation, UI, ...

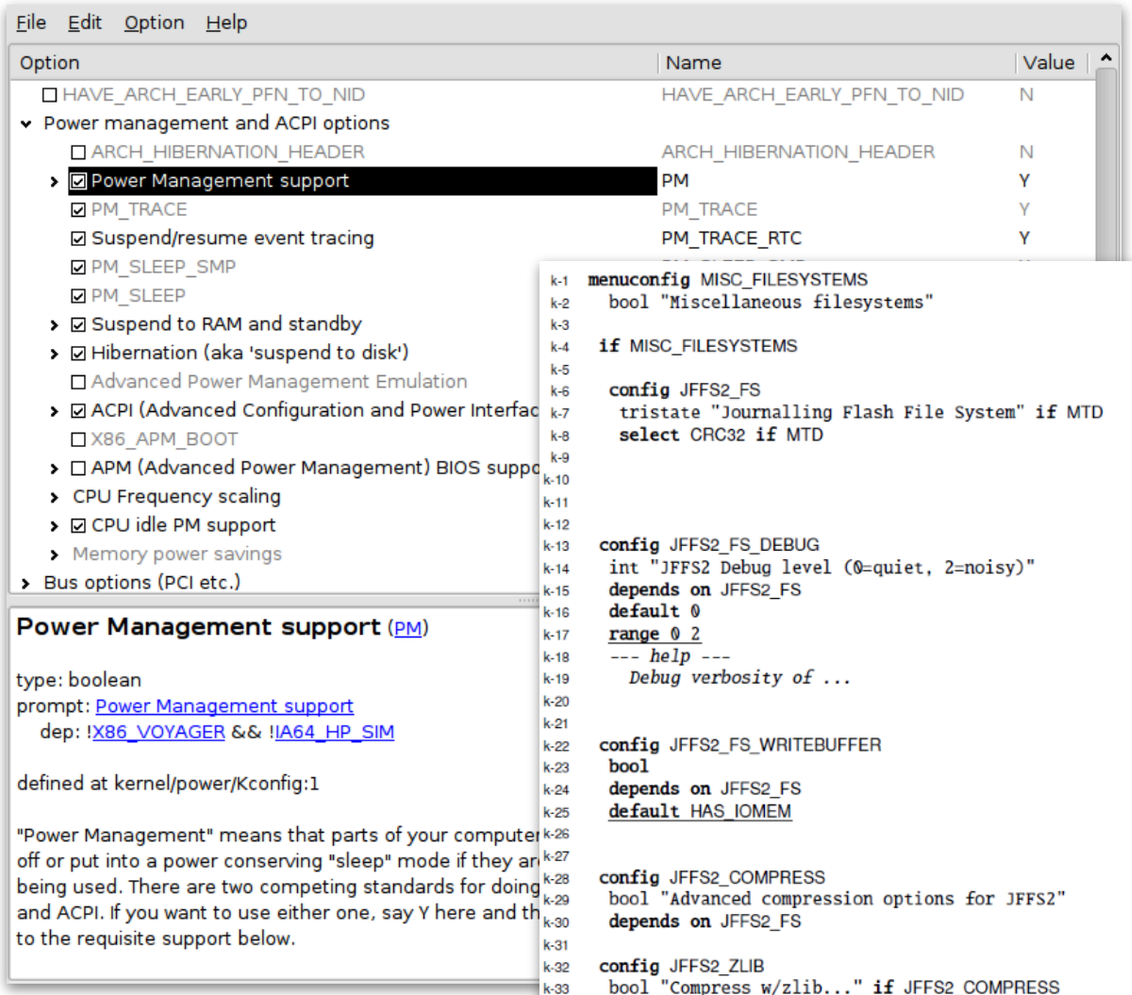




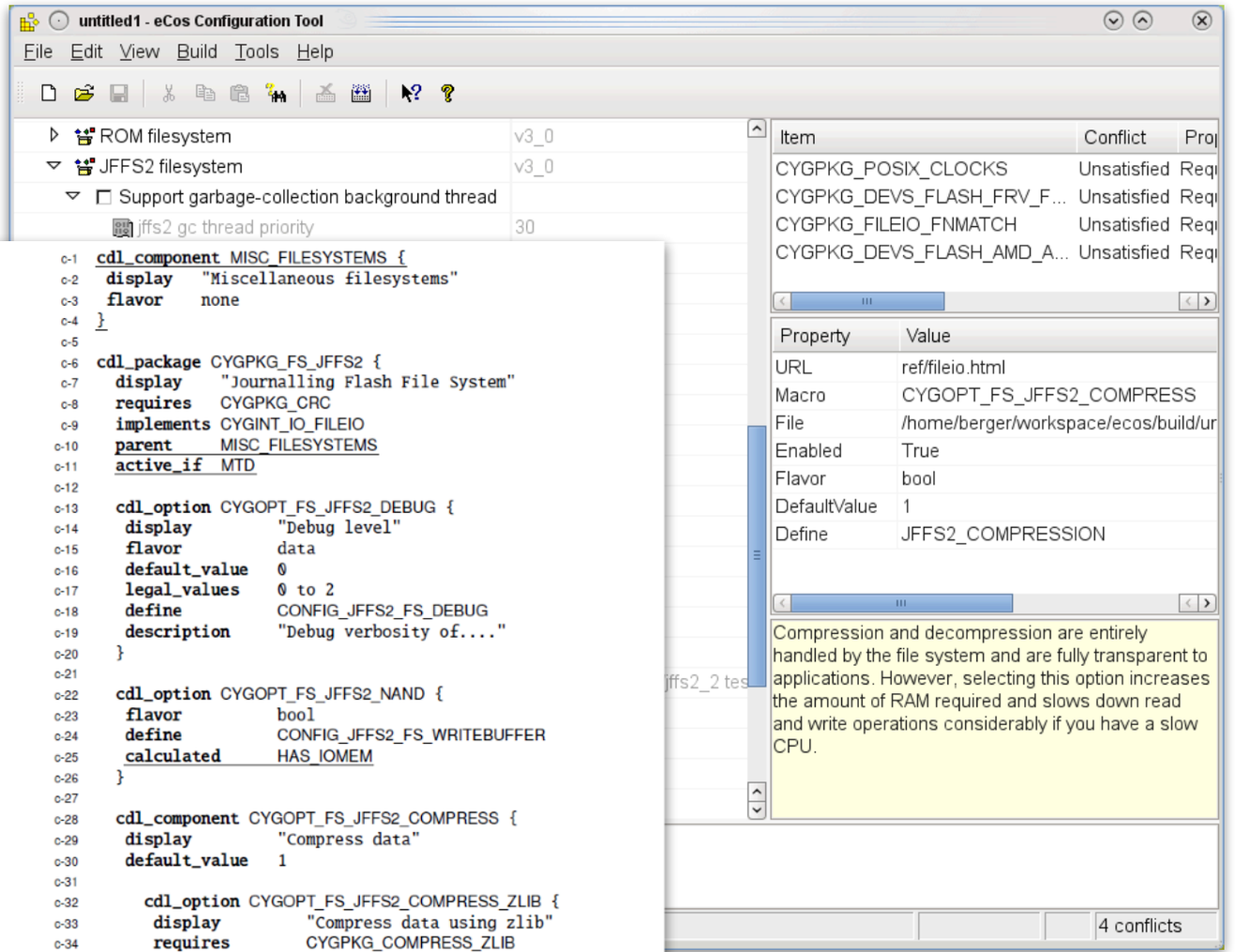


KConfig & CDL

Textual Variability Models




```
k-1  menuconfig MISC_FILESYSTEMS
k-2    bool "Miscellaneous filesystems"
k-3
k-4   if MISC_FILESYSTEMS
k-5
k-6    config JFFS2_FS
k-7     tristate "Journalling Flash File System" if MTD
k-8     select CRC32 if MTD
k-9
k-10
k-11
k-12
k-13   config JFFS2_FS_DEBUG
k-14    int "JFFS2 Debug level (0=quiet, 2=noisy)"
k-15    depends on JFFS2_FS
k-16    default 0
k-17    range 0 2
k-18    --- help ---
k-19      Debug verbosity of ...
k-20
k-21
k-22   config JFFS2_FS_WRITEBUFFER
k-23    bool
k-24    depends on JFFS2_FS
k-25    default HAS_IOMEM
k-26
k-27
k-28   config JFFS2_COMPRESS
k-29    bool "Advanced compression options for JFFS2"
k-30    depends on JFFS2_FS
k-31
k-32   config JFFS2_ZLIB
k-33    bool "Compress w/zlib..." if JFFS2_COMPRESS
k-34    depends on JFFS2_FS
k-35    select ZLIB_INFLATE
k-36    default y
k-37
k-38   choice
k-39    prompt "Default compression" if JFFS2_COMPRESS
k-40    default JFFS2_CMODE_PRIORITY
k-41    depends on JFFS2_FS
k-42    config JFFS2_CMODE_NONE
k-43     bool "no compression"
k-44    config JFFS2_CMODE_PRIORITY
k-45     bool "priority"
k-46    config JFFS2_CMODE_SIZE
k-47     bool "size (EXPERIMENTAL)"
k-48   endchoice
k-49  endif
```

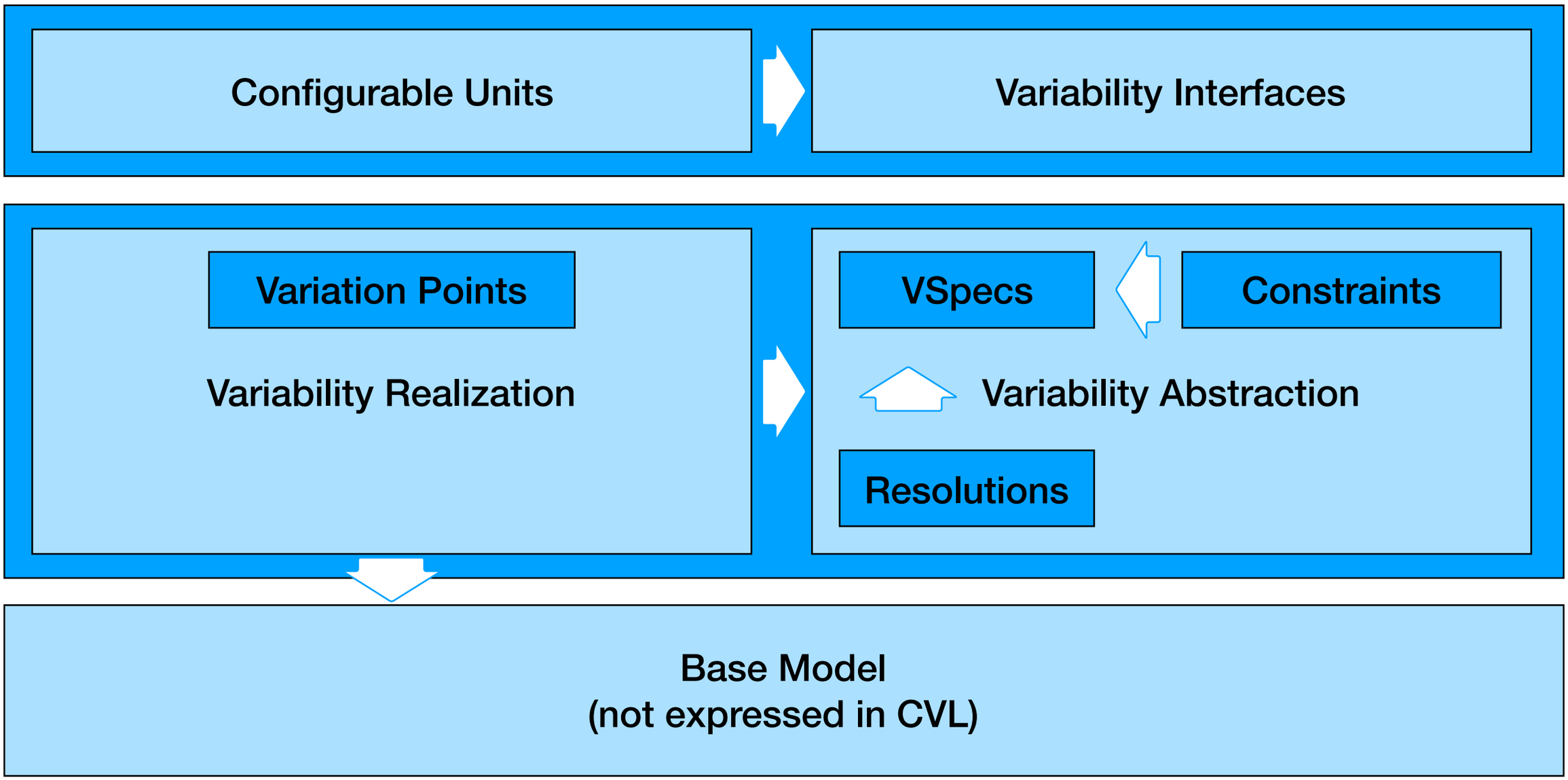



```
c-1  cdl_component MISC_FILESYSTEMS {
c-2   display    "Miscellaneous filesystems"
c-3   flavor     none
c-4  }
c-5
c-6  cdl_package CYGPKG_FS_JFFS2 {
c-7    display     "Journalling Flash File System"
c-8    requires    CYGPKG_CRC
c-9    implements  CYGINT_IO_FILEIO
c-10   parent      MISC_FILESYSTEMS
c-11   active_if   MTD
c-12
c-13   cdl_option CYGOPT_FS_JFFS2_DEBUG {
c-14    display          "Debug level"
c-15    flavor           data
c-16    default_value    0
c-17    legal_values     0 to 2
c-18    define           CONFIG_JFFS2_FS_DEBUG
c-19    description      "Debug verbosity of...."
c-20   }
c-21
c-22   cdl_option CYGOPT_FS_JFFS2_NAND {
c-23    flavor           bool
c-24    define           CONFIG_JFFS2_FS_WRITEBUFFER
c-25    calculated       HAS_IOMEM
c-26   }
c-27
c-28   cdl_component CYGOPT_FS_JFFS2_COMPRESS {
c-29    display          "Compress data"
c-30    default_value    1
c-31
c-32      cdl_option CYGOPT_FS_JFFS2_COMPRESS_ZLIB {
c-33       display          "Compress data using zlib"
c-34       requires         CYGPKG_COMPRESS_ZLIB
c-35       default_value    1
c-36      }
c-37
c-38      cdl_option CYGOPT_FS_JFFS2_COMPRESS_CMODE {
c-39       display          "Set the default compression mode"
c-40       flavor           data
c-41       default_value    { "PRIORITY" }
c-42       legal_values     { "NONE" "PRIORITY" "SIZE" }
c-43     }
c-44    }
c-45  }
c-46
c-47
c-48
c-49
```

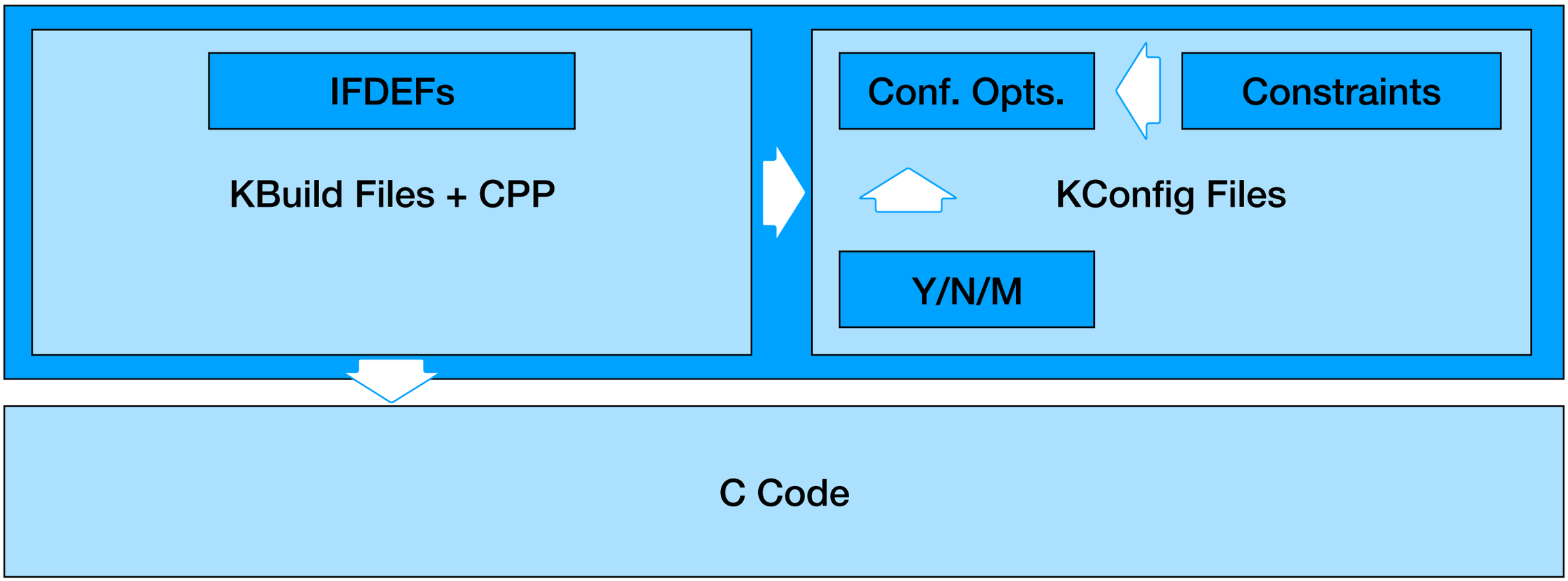


Common Variability Language (CVL)

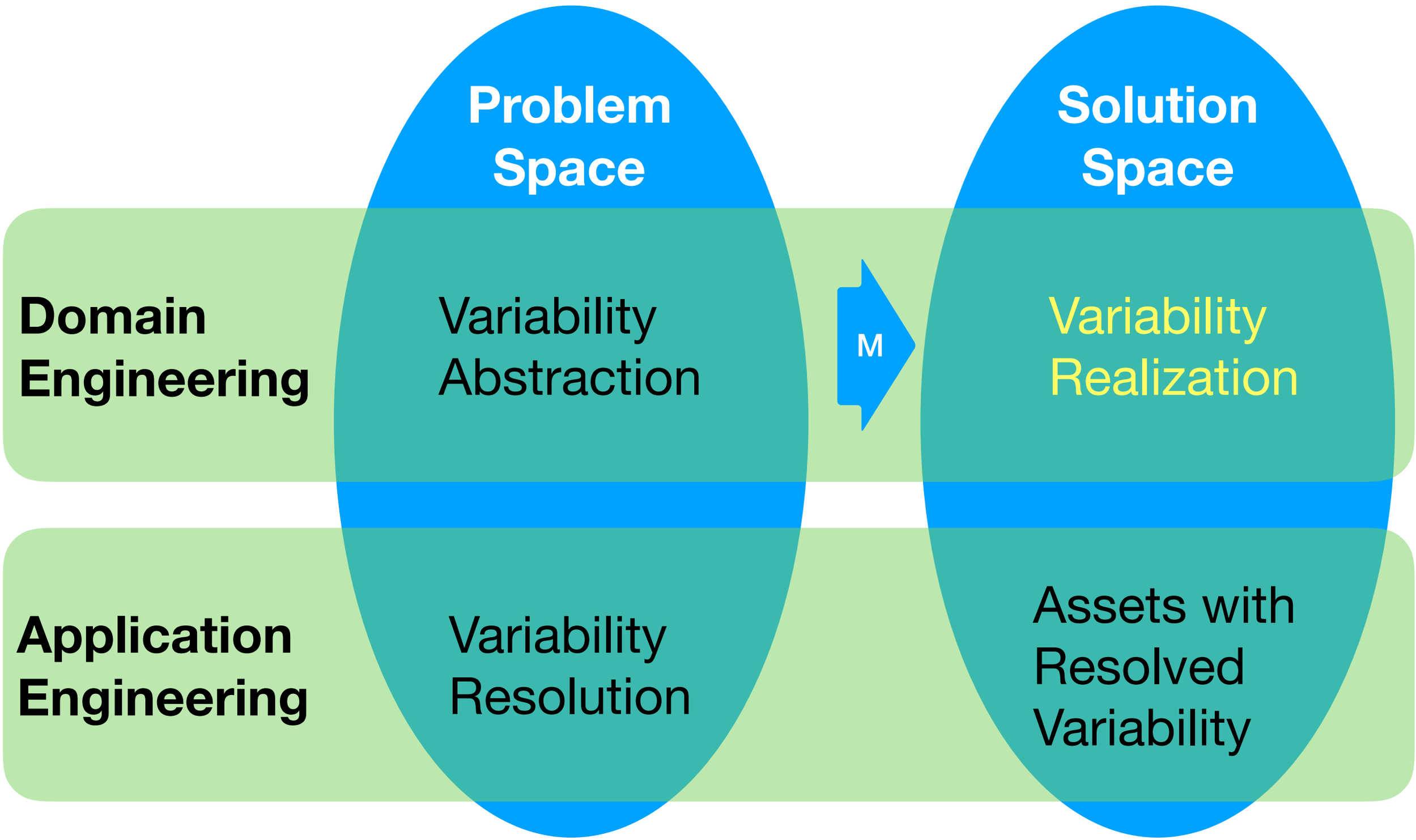
IBM et al., Proposal for CVL Revised Submission, 2012



CVL Architecture for Linux Junkies

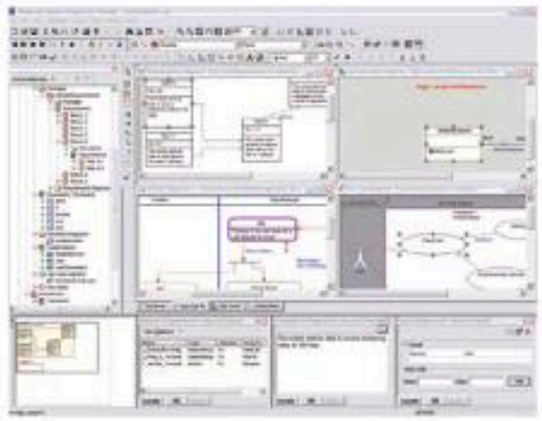


Software Product Line

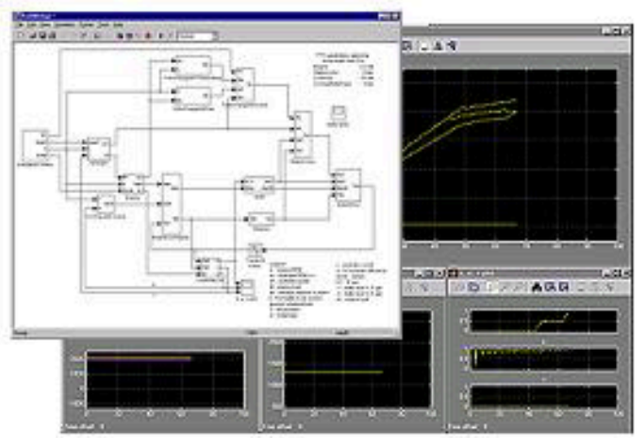


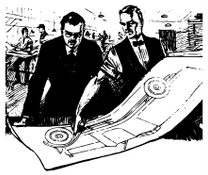
UML Models

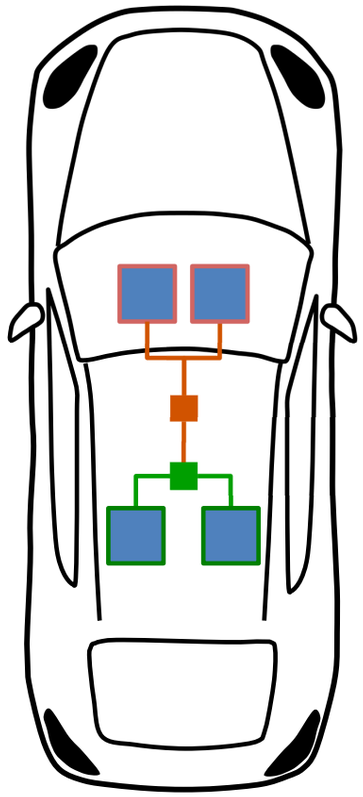




Hybrid Models





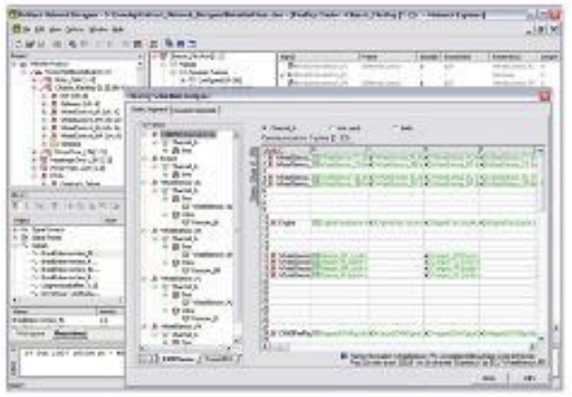


Calibrations



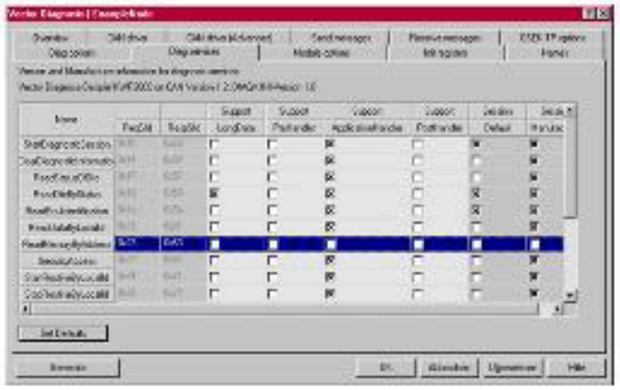


HW/SW Mapping



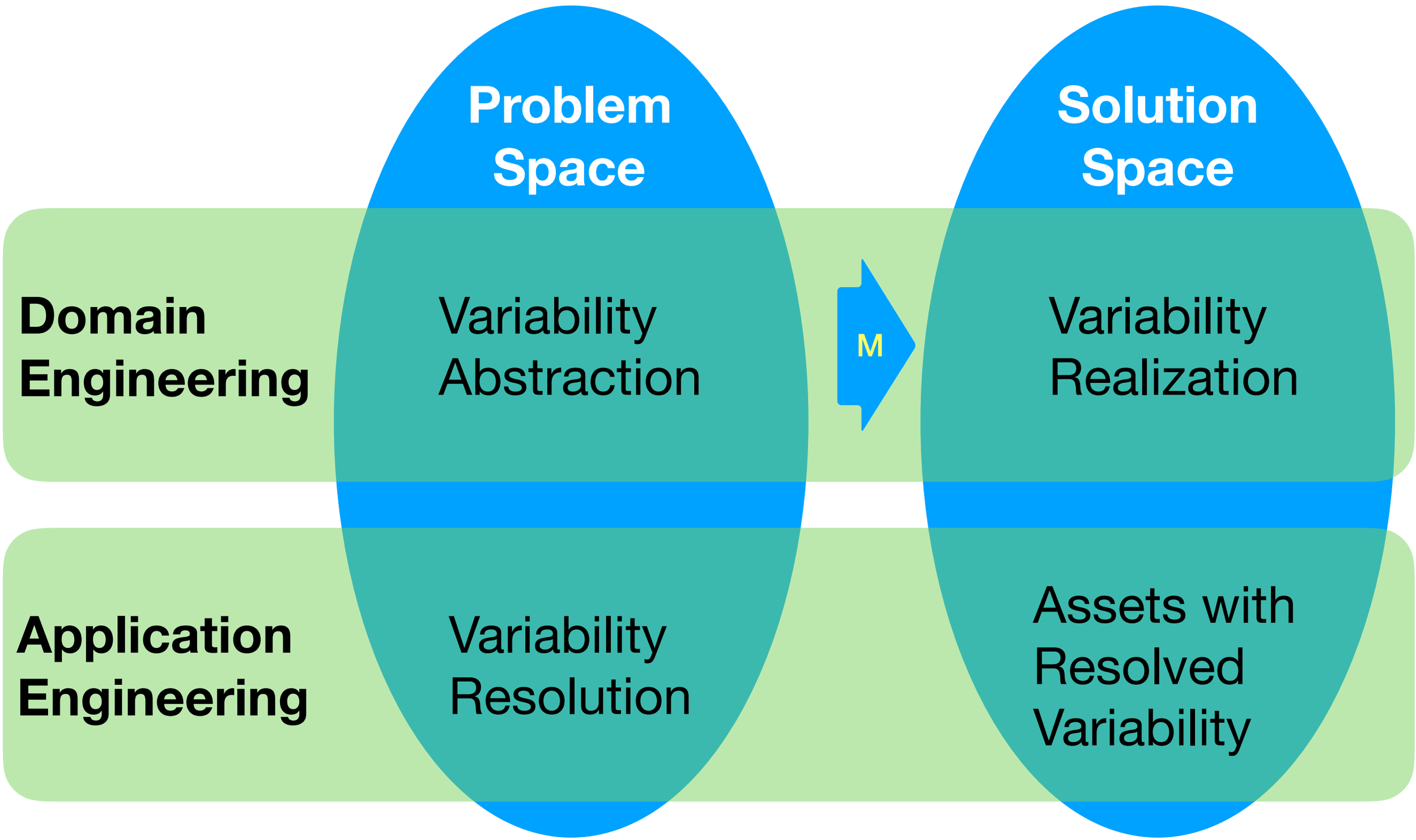


OS Generation



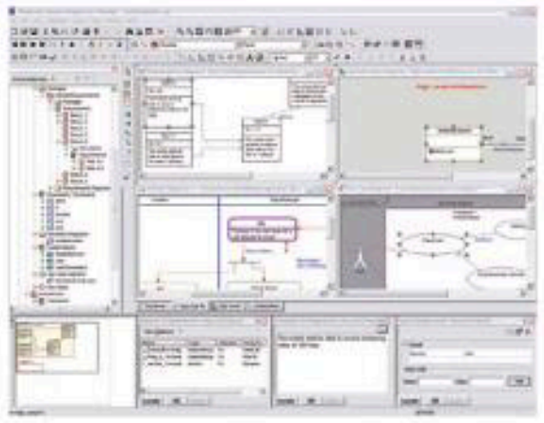


Software Product Line

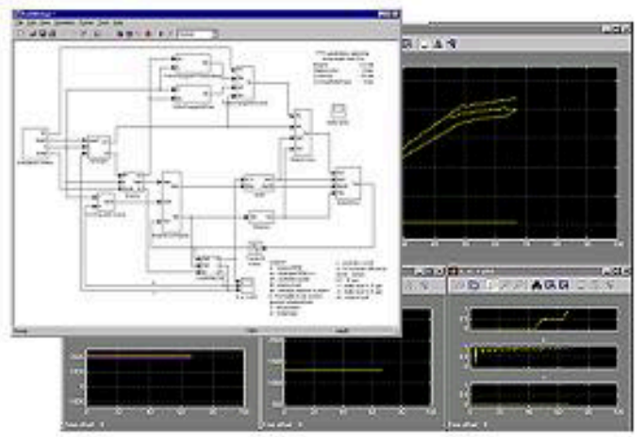


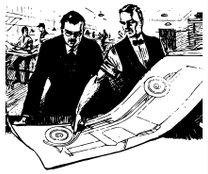
UML Models



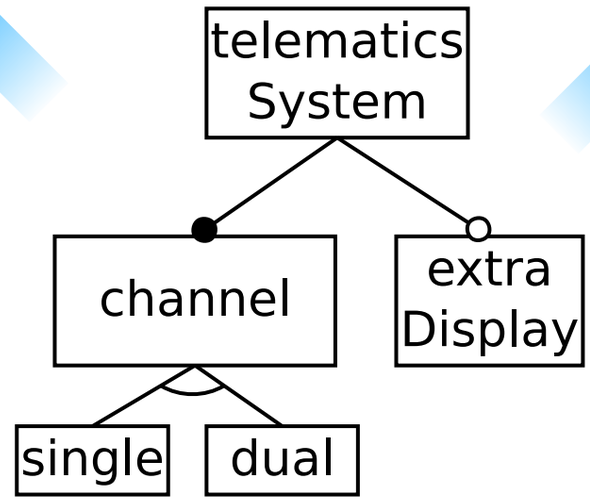


Hybrid Models





Features



M

M

M

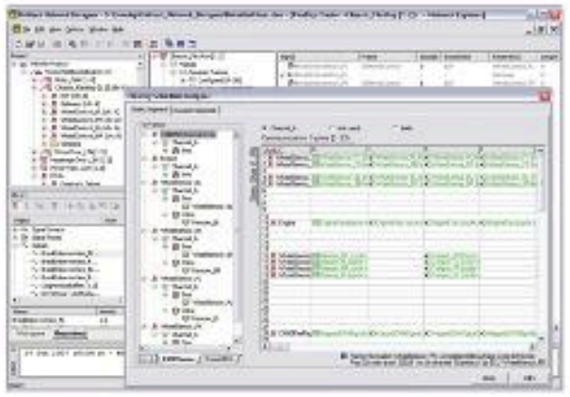
M

Calibrations



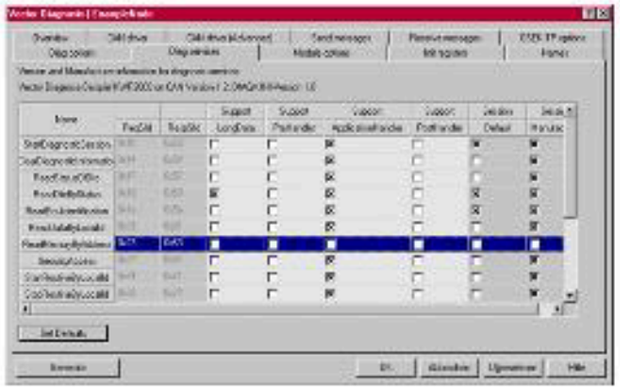


HW/SW Mapping



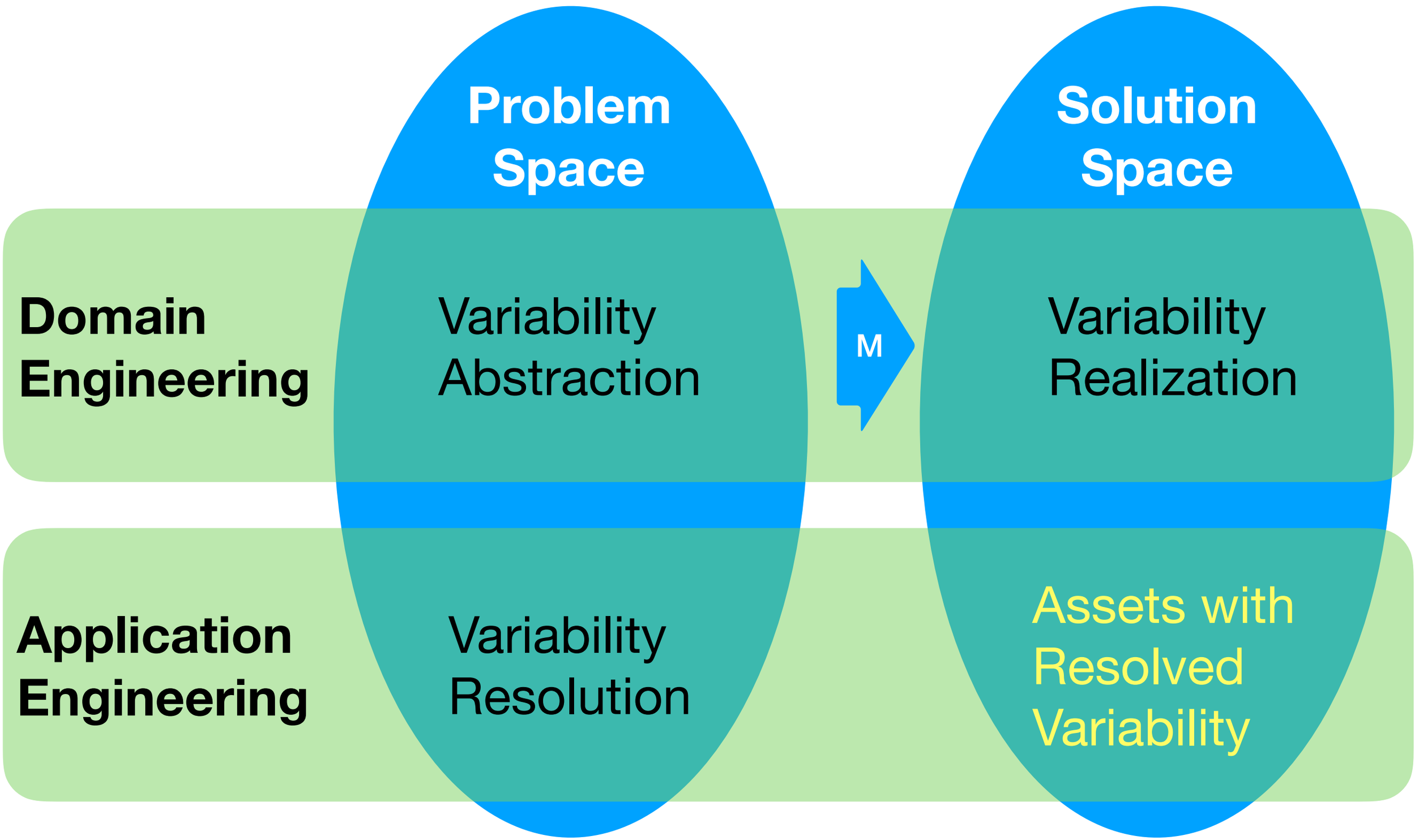


OS Generation

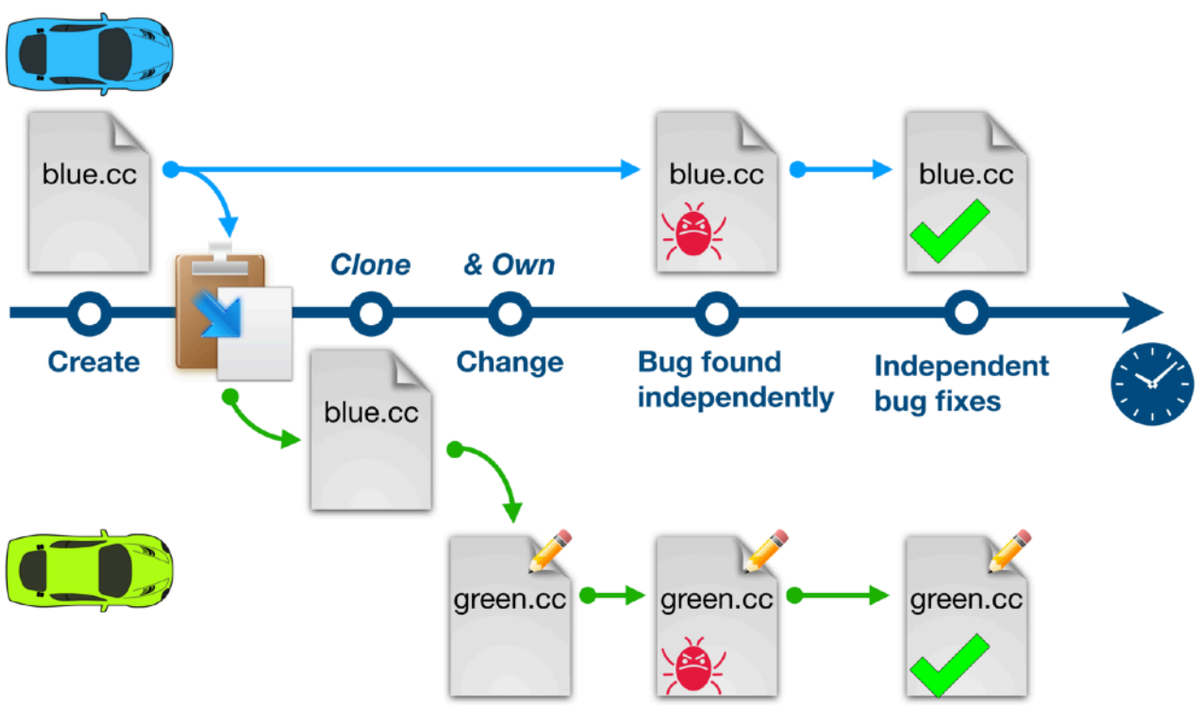




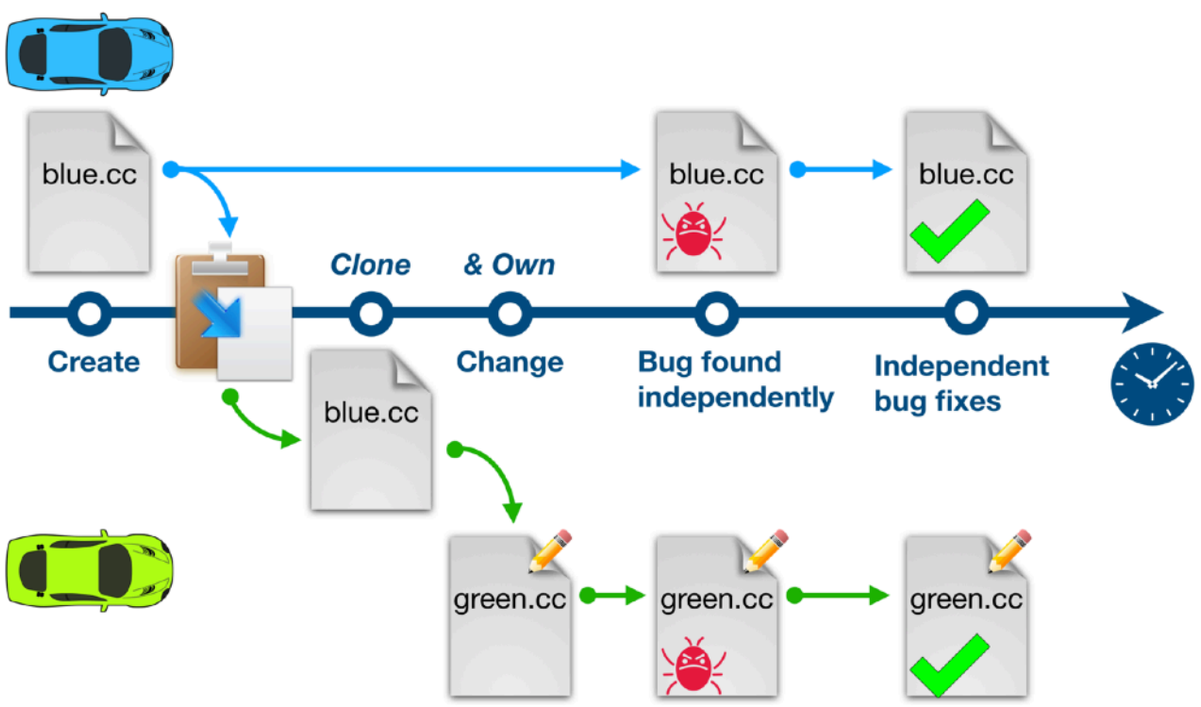
Software Product Line



Is Clone and Own Always Bad?

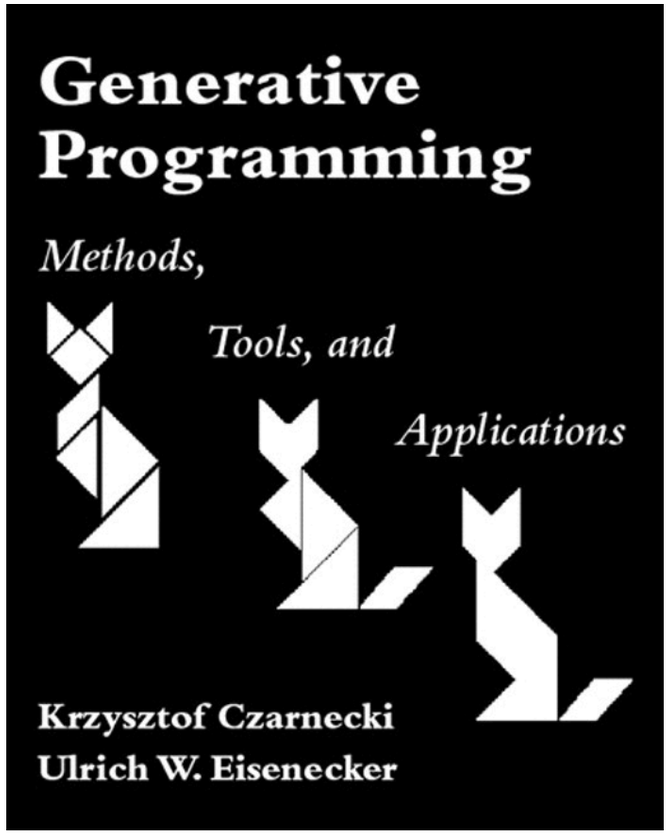


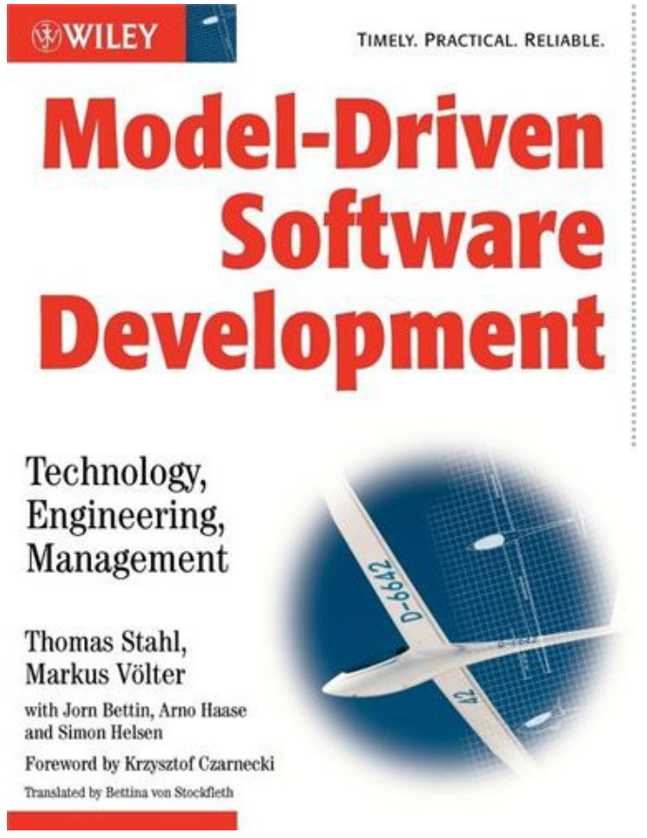
Is Clone and Own Always Bad?

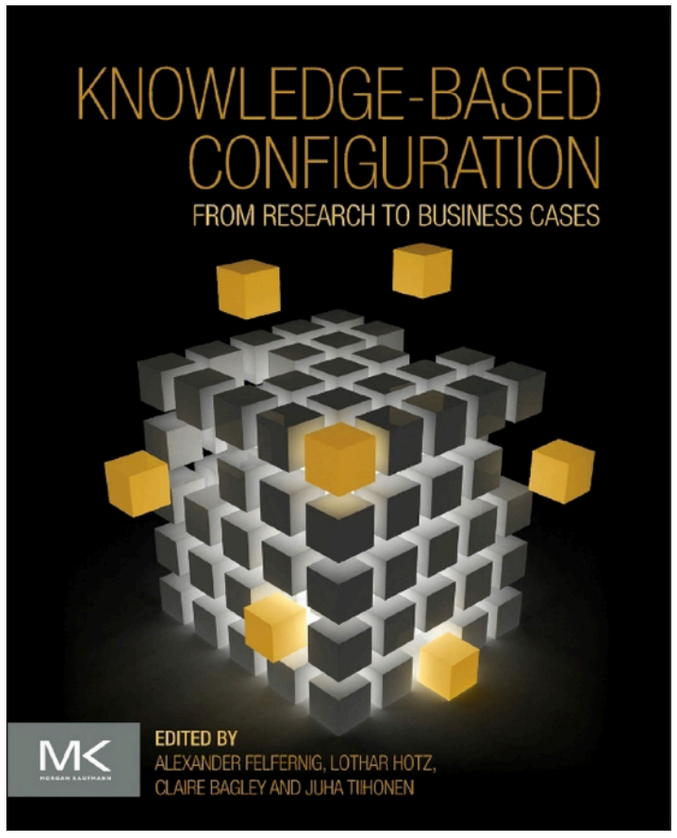


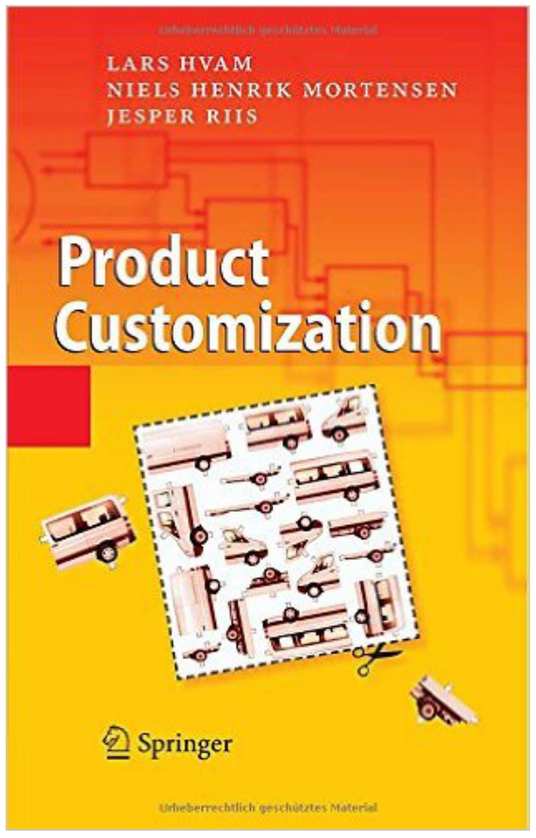
not if the cost of cloning is less

than the cost of an SPL 😉


Generative
Programming
Methods,
Tools, and
Applications
Krzysztof Czarnecki
Ulrich W. Eisenecker


WILEY
TIMELY. PRACTICAL. RELIABLE.
Model-Driven
Software
Development
Technology,
Engineering,
Management
Thomas Stahl,
Markus Völter
with Jorn Bettin, Arno Haase
and Simon Helsen
Foreword by Krzysztof Czarnecki
Translated by Bettina von Stockfleth


KNOWLEDGE-BASED
CONFIGURATION
FROM RESEARCH TO BUSINESS CASES
EDITED BY
ALEXANDER FELFERNIG, LOTHAR HOTZ,
CLAIRE BAGLEY AND JUHA TIIHONEN


LARS HVAM
NIELS HENRIK MORTENSEN
JESPER RIIS
Product
Customization
Springer
Urheberrechtlich geschütztes Material

Exercise

- Example Domain: *Traffic Lights*
  - Feature-oriented commonality/variability analysis
  - Domain concept analysis
  - Application configuration

- Apply *Example-Driven Modeling*

- Use Clafer & Web Tools
  - Tutorial style
  - Hands-on
  - Small exercises

Interactive Tutorial

http://t3-necsis.cs.uwaterloo.ca:8098/

Use Chrome or Firefox

Indent code with spaces, not tabs